

TALKTOTASTE – A VOICE CONTROLLED COOKING ASSISTANT

(Web Application Project)

A
PROJECT REPORT
ON
TALKTOTASTE
SUBMITTED BY
KHUSHI DINESH CHOUDHARY
UNDER THE GUIDENCE OF
PROFESSOR: SHRADDHA KOKATE
EXAMINATION BSC.IT, SEMESTER

University of Mumbai



UNIVERSITY OF MUMBAI
DEPARTMENT OF INFORMATION TECHNOLOGY
GHANSHYAMDAS SARAF COLLEGE OF ARTS AND COMMERCE
(Affiliated to university of Mumbai)
MUMBAI 400064
MAHARASHTRA
2025-2026

TALKTOTASTE

A Project Report

Submitted in complete fulfillment of the

Requirements for the award of the Degree of

BACHELOR OF SCIENCE (INFORMATION TECHNOLOGY)

By

KHUSHI DINESH CHOUDHARY

Roll No: 5

Under the esteemed guidance of

Prof. Shraddha Kokate



Ghanshyamdas Saraf
college of arts & commerce
EDUCATION EMPOWERS

DEPARTMENT OF INFORMATION TECHNOLOGY

GHANSHYAMDAS SARAF COLLEGE OF ARTS AND COMMERCE

(Affiliated to university of Mumbai)

MUMBAI 400064

MAHARASHTRA

2025-2026

PROFORMA FOR THE APPROVAL PROJECT PROPOSAL

(Note: All entries of the proforma of approval should be filled up with appropriate and complete information. Incomplete proforma of approval in any respect will be summarily rejected.)

PNR No:

Roll no: _____

1. Name of the Student

2. Title of the Project

3. Name of the Guide

4. Teaching experience of the Guide

5. Is this your first submission?

Yes

☐

No

☐

Signature of the Student

Signature of the Guide

Date:

Date:

Signature of the Coordinator

Date:

DEPARTMENT OF INFORMATION TECHNOLOGY
GHANSHYAMDAS SARAF COLLEGE OF ARTS AND COMMERCE

(Affiliated to university of Mumbai)



Ghanshyamdas Saraf
college of arts & commerce
EDUCATION EMPOWERS

CERTIFICATE

This is to certify that the project entitled, "**TalktoTaste**", is Bonafide work of **Khushi Choudhary** bearing Seat No: (5) submitted in complete fulfilment of the requirements for the award of degree of **BACHELOR OF SCIENCE in INFORMATION TECHNOLOGY** from University of Mumbai.

Internal Guide

Coordinator

External Examiner

Date:

College Seal

ROLES AND RESPONSIBILITIES

I, Khushi Dinesh Choudhary, solely take responsibility for the successful completion of the project titled “TalktoTaste – A Voice Controlled Cooking Assistant” within the given timeline. My role in this project involved the complete development of the system from scratch, including frontend, backend, database design, AI integration, testing, and documentation. All roles and responsibilities were fulfilled individually throughout the project lifecycle.

► Frontend Development

I was responsible for the complete frontend design and development of the TalktoTaste web application.

The key responsibilities included:

- Designed and developed a responsive user interface using Next.js, React, TypeScript, and Tailwind CSS.
- Created accessible and user-friendly layouts compatible with desktop, tablet, and mobile devices.
- Implemented a voice-first user interface to enable hands-free cooking interaction.
- Developed bilingual UI support for English and Hindi recipe narration.
- Designed screens such as Login, Dashboard, Recipe Display, Cooking Mode, Timer, Whistle Tracking, Profile, and Admin UI.
- Integrated Text-to-Speech (TTS) feedback with real-time UI updates.
- Ensured cross-browser compatibility and improved overall user experience through a clean and structured design.

► Backend Development

I independently handled the backend development and server-side logic of the application.

The responsibilities included:

- Developed RESTful APIs using Next.js server actions and Node.js.
- Implemented secure authentication and authorization using NextAuth.js and Google OAuth.
- Managed user sessions, roles, and access control.
- Developed backend logic for recipe management, user profiles, favorites, and cooking history.
- Implemented voice command intent processing for recipe navigation, timers, and whistle tracking.
- Designed and managed the Admin Dashboard backend for recipe and user management.
- Ensured secure data handling, validation, and error management.

► Database Design and Management

I was responsible for complete database design and data handling.

The responsibilities included:

- Designed database schema for Users, Recipes, Ingredients, Instructions, Timers, and Whistle Tracking.
- Implemented MongoDB for scalable and flexible data storage.
- Ensured data integrity, constraints, and proper relationships between collections.
- Managed CRUD operations for all system modules.
- Ensured secure and consistent storage of user and recipe data.

► AI, Voice Processing, and Smart Cooking Features

All AI-based and voice-controlled features were designed and implemented individually.

The responsibilities included:

- Integrated Speech-to-Text (STT) using Web Speech API.
- Implemented Text-to-Speech (TTS) for step-by-step recipe narration.
- Developed bilingual voice command recognition for Hindi and English.
- Implemented voice-controlled timers and reminders.
- Designed and implemented pressure cooker whistle tracking logic.
- Handled voice-command conflicts, fallback mechanisms, and speech interruptions.

► Testing and Quality Assurance

I independently performed all testing activities to ensure system reliability.

The responsibilities included:

- Designed and executed unit testing, integration testing, and system testing.
- Prepared functional and non-functional test cases.
- Tested voice recognition accuracy, bilingual support, timers, and whistle tracking.
- Verified system performance, usability, accessibility, and security.
- Debugged and resolved functional and voice-related issues.

► Documentation and Research

Complete documentation and research work were handled individually.

The responsibilities included:

- Conducted requirement analysis and technology survey.
- Designed UML diagrams such as DFD, Use Case, Activity, Sequence, Class, and ER diagrams.
- Prepared Gantt chart and project schedule.
- Compiled system design, implementation details, and testing reports.
- Prepared the final project report (Black Book) as per University of Mumbai guidelines.

ABSTRACT

TalktoTaste is an innovative voice-controlled web application developed to deliver a hands-free cooking experience using real-time speech interaction. The system allows users to search for recipes, follow step-by-step cooking instructions, and perform cooking-related tasks through simple voice commands. By integrating speech recognition and text-to-speech technologies, the application enables smooth control of recipe navigation through commands such as “next,” “repeat,” and timer instructions without requiring physical interaction with the device. The platform supports bilingual communication in English and Hindi, ensuring accessibility for a broader group of users. Features such as voice-based timers, pressure cooker whistle monitoring, recipe storage, and user profile management improve usability and personalization. An admin dashboard is provided to manage recipes and user information efficiently. Built using modern web technologies including Next.js and MongoDB, TalktoTaste delivers a responsive, secure, and user-friendly interface. The system reduces dependency on traditional screen-based recipe applications and introduces an intelligent voice-first kitchen solution that enhances convenience, safety, and accessibility during cooking.

Keywords:

- Voice-Controlled Cooking Assistant
- Speech Recognition
- Text-to-Speech (TTS)
- Hands-Free Cooking
- Bilingual Support (Hindi & English)
- Recipe Navigation
- Voice-Based Timer
- Whistle Tracking System
- User Profile Management
- Web-Based Application

ACKNOWLEDGEMENT

I take this opportunity to express my profound gratitude and indebtedness to my project guides Prof. Shraddha Kokate for giving me the opportunity to accomplish this project.

I am very much thankful to Prof. Shraddha Kokate for being very much resourceful, kind and helpful. Their positive attitude, unassailable optimism and unwavering faith in me assured that I came out of problems whenever I encountered difficulties.

Finally I wish to thank all my friends and IT department to directly or indirectly helped me in completion of this project. Last but not the least I would thank my family without whose support, motivation and encouragement this would not have been possible.

TABLE OF CONTENT

CHAPTER 1: INTRODUCTION	1
1.1 Background	1
1.2 Objective	1
1.3 Purpose, Scope and Applicability	2
1.3.1 Purpose	2
1.3.2 Scope	2
1.3.3 Applicability	3
1.5 Organization of Report	3
CHAPTER 2: SURVEY OF TECHNOLOGY	4
2.1 Survey of Programming Languages	4
2.2 Survey of AI Algorithms and Modules	5
2.3 Survey of Databases and Data Storage Solutions	6
2.4 Survey of Design Software	7
2.5 Survey of Frontend Technologies	8
2.6 Survey of Backend Technologies	9
2.7 Survey of Development Tools	10
2.8 Survey of Version Control Systems	11
2.9 Survey of Deployment Infrastructure	12
2.10 Survey of Project Management Software	13
CHAPTER 3: REQUIREMENT AND ANALYSIS	14
3.1 Problem Definition	14
3.2 Requirement Specification	14
3.3 Planning and Scheduling	15
3.4 Hardware Requirement	16
3.5 Software Requirement	16
3.6 Conceptual Models	16
3.6.1 Data Flow Diagram	17
3.6.2 Use Case Diagram	19
3.6.3 Activity Diagram	20
3.6.4 Sequence Diagram	21
3.6.5 Class Diagram	22
3.6.6 Entity Relationship Diagram	23
3.6.7 Event Table	25

CHAPTER 4: SYSTEM DESIGN	26
4.1 Basic Modules	26
4.2 Data Design	26
4.2.1 Schema Design	26
4.2.2 Data Integrity	28
4.3 Procedure Design	31
4.3.1 Flow Chart	31
4.4 Algorithm Design	32
4.5 UI Design	33
4.6 Test Case Design	35
CHAPTER 5: IMPLEMENTATION AND TESTING	36
5.1 Implementation Approaches	36
5.2 Coding Details and Code Efficiency	36
5.2.1 Coding	36
5.2.2 Coding Efficiency	39
5.3 Testing Approach	40
5.3.1 Unit Testing	40
5.3.2 Integration Testing	41
5.3.3 Beta Testing	42
5.4 Modification and Improvements	43
5.5 Test Cases	44
CHAPTER 6: RESULTS AND DISCUSSION	45
6.1 Test Reports	45
6.2 User Documentation	47
CHAPTER 7: CONCLUSIONS	51
7.1 Conclusion	51
7.1.1 Significance of the System	52
7.2 Limitations of the System	53
7.3 Future Scope of the Project	54
REFERENCES	54

LIST OF TABLES

Table 1 Comparison table for Programming languages.....	4
Table 2 Comparison Table for AI Algorithm	5
Table 3 Comparison Table for database and data storage solutions	6
Table 4 Comparison Table for design software	7
Table 5 Comparison Table for Frontend Technologies.....	8
Table 6 Comparison Table for backend Technologies.....	9
Table 7 Comparison Table for development tools	10
Table 8 Comparison Table for version controls	11
Table 9 Comparison Table for Deployment Infrastructure	12
Table 10 Comparison Table for project management software	13
Table 11 Planning and scheduling table.....	14
Table 12 Event Table	25
Table 13 Schema Design	26
Table 14 Data Integrity Design.....	28
Table 15 Test Case	35
Table 16 Unit Testing	40
Table 17 Integration Testing	41
Table 18 Beta Testing	42
Table 19 Test Cases	44

LIST OF FIGURES

Fig 1: Gantt Chart.....	15
Fig 2: Pert Chart	15
Fig 3: Data Flow Diagram	17
Fig 4: Use Case Diagram	19
Fig 5: Activity Diagram	20
Fig 6: Sequence Diagram	21
Fig 7: Class Diagram.....	22
Fig 8: Entity Relation Diagram	24
Fig 9: FlowChart.....	31

CHAPTER 1

INTRODUCTION

1.1 Background

Cooking is an essential activity in every household. However, interacting with digital devices in a kitchen environment often proves inconvenient and unhygienic. Many people especially beginners, elderly individuals, and people with physical disabilities find it difficult to follow recipes on smartphones or tablets while handling ingredients or working with messy hands.

- Messy or wet hands make it hard to scroll or operate touch-based screens. - Most recipe formats such as apps, books, or videos demand continuous visual and physical attention.
- Touch-based interfaces are impractical in real-time cooking situations.
- Users seek a hands-free, voice-operated solution for navigating cooking steps.
- Recent studies indicate:

Over 65% of individuals use mobile devices to follow recipes while cooking.

More than 70% report frustration when trying to interact with screens mid-cooking. A growing number of users, particularly those with mobility or vision limitations, express preference for voice-assisted cooking tools.

This presents a clear opportunity to design a smart, inclusive kitchen assistant that delivers real-time recipe guidance through voice interaction, reducing manual effort and enhancing accessibility.

TalktoTaste is designed to meet this need.

It is a voice-controlled cooking assistant that guides users through Indian recipes in a hands-free manner. Supporting both Hindi and English, the assistant offers a safe, intuitive, and accessible way to cook especially tailored for Indian households.

1.2 Objective

The main objectives of TalktoTaste are as follows:

- **develop a voice-first smart kitchen assistant** that enables voice control recipe interaction for Indian cooking.
- **To offer step-by-step audio instructions** in Hindi and English, ensuring accessibility across different language preferences.
- **Allow users to search for recipes using voice commands**, either by dish name or by specifying available ingredients.
- **Integrate voice-activated timers and reminders**, enhancing time management during cooking.
- **promote inclusivity**, especially for elderly individuals, physically disabled users, and beginners in the kitchen.

- **To support a safer and cleaner cooking environment** by reducing the need to touch screens or devices everytime.
- **Implement cooking duration and whistle tracking**, allowing users to manage pressure cooking with voice inputs.
- **serve as a personal cooking assistant**, simplifying daily meal preparation through intelligent voice- guided interactions.

1.3 Purpose and Scope

1.3.1 Purpose

The purpose of TalktoTaste is to create a hands-free, voice-powered kitchen experience tailored for Indian home cooks. It addresses common frustrations associated with screen-based recipe formats and aims to provide a comfortable, accessible, and judgment-free cooking environment.

This assistant is especially valuable for:

- Users who prefer not to touch devices while cooking
- Elderly or physically challenged individuals who require voice-based assistance
- People who want bilingual recipe instructions in Hindi and English
- Beginners who benefit from detailed, guided steps
- Anyone seeking an inclusive, voice-first cooking experience

1.3.2 Scope

TalktoTaste offers a range of features to support a seamless, touchless cooking experience. The scope includes:

- **Supports voice interaction:** Users can control the assistant, search for recipes, and receive spoken instructions—all hands-free.
- **Step-by-step recipe narration:** The assistant walks the user through each recipe using clear voice instructions in Hindi and English.
- **Voice-based search:** Recipes can be found by simply stating the dish name or listing available ingredients.
- (e.g., “How to make Paneer Butter Masala”) or by mentioning available ingredients (e.g., “What can I cook with rice and potato?”).
- **Timer and reminder features:** Users can set and manage timers using voice commands without interrupting their cooking.
- **Recipe saving and sharing:** Voice-based recipe storage and sharing options promote a hygienic and efficient workflow.
- **Accessibility support:** Designed with various user groups in mind—such as the elderly, disabled, or visually impaired.
- **Bilingual content:** Both Hindi and English are supported in the current version, with plans to expand to regional languages.

- **YouTube video integration:** For complex dishes, users can request video tutorials via voice or manual selection.
- **Whistle and cooking time tracking:** The assistant can track whistle counts and cooking durations based on voice inputs like “Started now” or “2 whistles done.”

Example:

For Rajma, the assistant says:

“Let it whistle 3 times. Tell me when it starts.”

User responds: “Started now” → “2 whistles done”

Assistant replies: “One more to go. I’ll notify you after that.”

1.3.3 Applicability

TalktoTaste serves a wide range of users and use cases:

Where it can be used:

- **Home Kitchens:** A personal tool that supports daily cooking.
- **Elderly and Disabled Users:** Offers an easy, accessible way to follow recipes. - **Smart Kitchens:** Can be integrated with IoT devices for enhanced functionality for future plan.
- **Busy Professionals:** Allows multitasking while cooking with voice-controlled guidance.
- **Students and Beginners:** Provides step-by-step support while learning to cook Indian dishes.
- **Who can benefit:**
- **Home cooks:** For routine meal preparation.
- **Elderly and physically challenged users:** Enables independence in cooking.
- **Beginners and students:** Provides guided, easy-to-follow instructions.
- **Busy professionals:** Supports multitasking with voice-based operation.
- **Caregivers:** Helps guide individuals with special needs.
- **Non-English speakers:** Hindi support ensures wider accessibility.

1.5 Organization of Report

The subsequent chapters of the project report will focus on the survey of different technologies which includes comparison of different platforms, languages, software, hardware etc.

The Survey is followed by the analysis of requirements resulting in generation of requirement specification and schedule of activities. It also includes the conceptual design that visualizes the features and operations that can be performed on the system.

The final chapter of the project report includes the system design that describes desired features and operations in detail, including screen layouts, business rules, process diagrams, pseudo code and other documentation.

CHAPTER 2

SURVEY OF TECHNOLOGY

2.1 Survey of Programming Languages

a) List of Technologies Considered

- Python – Beginner-friendly language with strong AI/ML support.
- JavaScript – Widely used for both frontend and backend development (Node.js + Next.js).
- TypeScript – Superset of JavaScript with static typing for scalable and reliable web applications.

b) Comparison

Language	Platform Support	Performance	Ease of Learning	Community Support	Remarks
Python	Backend, AI/ML	Moderate	Very Easy	Very Strong	Ideal for AI, data processing & voice logic
JavaScript	Frontend + Backend (Next.js)	High	Easy	Very Strong	Excellent for interactive applications
TypeScript	Frontend + Backend (via JS)	High	Moderate	Strong	Best for scalable and bug-free web apps

Table 1: Comparison table for Programming languages

c) Why to Choose

Python

- Provides powerful AI/ML libraries (TensorFlow, PyTorch, NLTK).
- Simple and readable syntax speeds up development.
- Works on multiple platforms (Windows, Linux, Mac).
- Supported by a large developer community.

JavaScript

- Enables full-stack development (Next.js for backend, React for frontend).
- Supports real-time interactions essential for voice-based apps.

- Backed by a huge ecosystem of frameworks and libraries.
- Provides seamless API and database integration.

TypeScript

- Adds static typing on top of JavaScript, reducing runtime errors.
- Ideal for building Progressive Web Apps (PWAs) with reliability.
- Scales better for long-term projects by improving code maintainability.
- Works seamlessly with React (frontend) and Node.js + Next.js (backend).
- Widely used in modern industry (Microsoft, Slack, Airbnb).

2.2 Survey of AI Algorithms and Modules

a) List of Technologies Considered

- Speech-to-Text (STT) – Converts spoken words into text.
- Text-to-Speech (TTS) – Converts text into natural-sounding speech.
- Natural Language Processing (NLP) – Understands Hindi/English commands.
- Recommendation Algorithms – Suggests recipes based on context.

b) Comparison

Algorithm/Module	Accuracy	Complexity	Use Case
Speech-to-Text	High	Moderate	Converts user voice to text
Text-to-Speech	High	Low	Reads instructions aloud
NLP	High	High	Understands bilingual commands
Recommendation	High	Moderate	Suggests recipes to users

Table 2: Comparison table for Algorithms

c) Why to Choose

Speech-to-Text

- Ensures smooth voice command recognition.
- Enhances accessibility during cooking.

Text-to-Speech

- Provides hands-free recipe narration.
- Improves usability in noisy or busy kitchens.

Natural Language Processing

- Enables bilingual interaction (Hindi & English).
- Understands intent and context in user queries.

Recommendation Algorithms

- Personalizes recipe suggestions.
- Adapts to user preferences and available ingredients.

2.3 Survey of Databases and Data Storage Solutions

a) List of Technologies Considered

- MongoDB – NoSQL, JSON-like documents.
- MySQL – Relational database.
- SQLite – Lightweight, file-based.

b) Comparison

Database	Type	Scalability	Offline Support	Best For	Cost Structure
MongoDB	NoSQL	Very High	Limited	JSON data, cloud apps	Free/Paid
MySQL	Relational	High	No	Structured data apps	Free/Paid
SQLite	Relational	Low	Yes	Lightweight local apps	Free

Table 3: Comparison table for Databases

c) Why to Choose

MongoDB

- Flexible document-based storage.
- Highly scalable for cloud applications.

- Integrates seamlessly with Node.js + Next.js.

MySQL

- Reliable relational system with ACID compliance.
- Excellent for structured recipe/user data.
- Widely supported in industry.

SQLite

- Perfect for lightweight apps and prototyping.
- Provides offline support with no external server.

2.4 Survey of Design Software

a) List of Tools Considered

- Figma – Cloud-based design and prototyping tool.
- Adobe XD – UI/UX design software.
- Canva – Simple online design tool.

b) Comparison

Tool	Collaboration	Ease of Use	Best For
Figma	Excellent	Easy	Team-based UI design
Adobe XD	Good	Moderate	Professional prototyping
Canva	Limited	Very Easy	Quick graphics and mockups

Table 4: Comparison table for list of Design Tools

c) Why to Choose

Figma

- Enables real-time team collaboration.
- Free tier supports most project needs.
- Accessible across devices via browser.

Canva

- Extremely beginner-friendly.
- Useful for non-technical design needs.

2.5 Survey of Frontend Technologies

a) List of Technologies Considered

- HTML, CSS, JavaScript – Core web stack.
- Bootstrap / Tailwind CSS – Styling frameworks.
- React.js – Component-based library.

b) Comparison

Technology	Performance	UI Quality	Community Support	Best For
HTML/CSS/JS	High	Basic	Very Strong	Core web apps
Bootstrap/Tailwind	High	Good	Strong	Responsive design
React.js	Very High	Excellent	Very Strong	Dynamic web apps

Table 5: Comparison table for Frontend languages

c) Why to Choose

HTML / CSS / JS

- Universal and browser-compatible.
- Foundation for all web applications.

Bootstrap / Tailwind CSS

- Pre-built UI components.
- Ensures mobile-first responsive design.

React.js

- Reusable UI components.
- Fast rendering with Virtual DOM.
- Backed by strong developer community.

2.6 Survey of Backend Technologies

a) List of Technologies Considered

- Python (Flask/Django).
- Node.js + Next.js.
- REST APIs.

b) Comparison

Technology	Performance	Scalability	AI Integration	Best For
Flask/Django	High	High	Excellent	AI&NLP apps
Node.js/Next.js	High	High	Medium	Real-time scalable apps
REST APIs	High	High	Medium	Cross-platform comms

Table 6: Comparison table for Backend languages

c) Why to Choose

Flask/Django

- Strong AI/ML support.
- Quick development and integration.

Node.js + Next.js

- Event-driven architecture.
- Works best with MongoDB.
- Scales well for API-driven apps.

REST APIs

- Universal communication standard.
- Ensures interoperability.

2.7 Survey of Development Tools

a) List of Tools Considered

- Visual Studio Code.
- PyCharm.
- Postman.

b) Comparison

Tool	Best For	Ease of Use	Cost
VS Code	Multi-language coding	Easy	Free
PyCharm	Python development	Moderate	Free/Paid
Postman	API testing	Easy	Free

Table 7: Comparison table for Development tools

c) Why to Choose

Visual Studio Code

- Lightweight and highly extensible.
- Supports multiple languages.

PyCharm

- Best for Python-heavy AI development.
- Provides debugging and project management.

Postman

- Simplifies API testing.
- Automates request/response validation.

2.8 Survey of Version Control Systems

a) List of Tools Considered

- Git.
- GitHub.

b) Comparison

Tool	Hosting	Collaboration	Cost
Git	Local	Limited	Free
GitHub	Cloud	Excellent	Free/Paid

Table 8: Comparison table for Version Control tools

c) Why to Choose

Git

- Reliable local version control.
- Industry-standard tool.

GitHub

- Simplifies collaboration and code sharing.
- Offers free repositories.

2.9 Survey of Deployment Infrastructure

a) List of Platforms Considered

- Local Machine.
- Heroku.
- Render.
- Vercel.
- Netlify.
- Docker.

b) Comparison

Platform	Scalability	Ease of Use	Best For
Local Machine	Low	Easy	Development/testing
Heroku	High	Easy	Small apps, prototyping
Render	High	Easy	APIs and backend apps
Vercel	High	Very Easy	React/Next.js apps
Netlify	High	Very Easy	Static websites
Docker	Very High	Moderate	Containerized deployments

Table 9: Comparison table for Development Infrastructure platform

c) Why to Choose

Heroku

- Simple Git-based deployment.
- Good for prototypes.

Render

- Modern alternative to Heroku.
- Supports free tier apps.

Vercel

- Specializes in frontend hosting.
- Free plan supports serverless APIs.

Docker

- Ensures consistency across environments.
- Ideal for scalable cloud deployments.

2.10 Survey of Project Management Software

a) List of Tools Considered

- **Trello** – Visual task boards for simple project tracking.
- **ClickUp** – All-in-one productivity and project management tool.
- **Notion** – Notes, documentation, and lightweight task management in one platform.

b) Comparison

Tool	Best For	Ease of Use	Cost	Relevance to TalktoTaste
Trello	Simple Kanban-style boards	Very Easy	Free	Manage recipe module tasks, bug tracking
ClickUp	Agile workflows & integrations	Easy	Free/Paid	Track AI/ML model development, deadlines
Notion	Docs + tasks in one workspace	Easy	Free/Paid	Maintain recipe datasets, documentation, and sprints

Table 10: Comparison table for project management

c) Why to Choose

For TalktoTaste, **Trello** is chosen for its **simple visual boards** that streamline recipe module tasks and bug tracking, **ClickUp** supports **agile workflows and integrations** making it effective for managing **AI/ML model development** and deadlines, while **Notion** provides a unified space for **documentation, datasets, and sprint planning**. Together, these tools ensure **clarity, productivity, and collaboration** across the project lifecycle.

CHAPTER 3

REQUIREMENT AND ANALYSIS

3.1 Problem Definition

Cooking while hands-busy is inconvenient: users can't easily use touch-based recipe apps. Many apps require manual interaction, causing interruptions and mistakes.

Users forget steps, timings or quantities leads to overcooking / wrong measurements. Need for real-time, hands-free guidance with natural voice commands and clear spoken feedback.

TalktoTaste = a voice-controlled assistant that listens, understands cooking commands, and speaks step-by-step instructions so users can cook hands-free.

3.2 Requirement Specification

- Voice Commands — accept spoken commands: “next”, “repeat”, “Start timer”.
- Speech Recognition — convert user speech → actionable intents.
- Text-to-Speech (TTS) — read recipe steps aloud.
- Recipe Search — search by name, ingredient, cuisine.
- Step Navigation — next step, previous step, repeat, jump to step n.
- Timers — set/manage timers by voice.
- Favourite / Profile — save favourite recipes and user preferences.
- Usability — clear spoken instructions.
- Availability — usable on smartphones & browsers.
- Privacy & Security — user data encrypted.
- Scalability — support growing number of recipes.
- Localization — support multiple languages like Hindi and English.

3.3 Planning and Scheduling

Number of Personnel: 1

Estimated Start Date: 17th July 2025

Estimated End Date: 28th Feb 2026

Gantt Chart

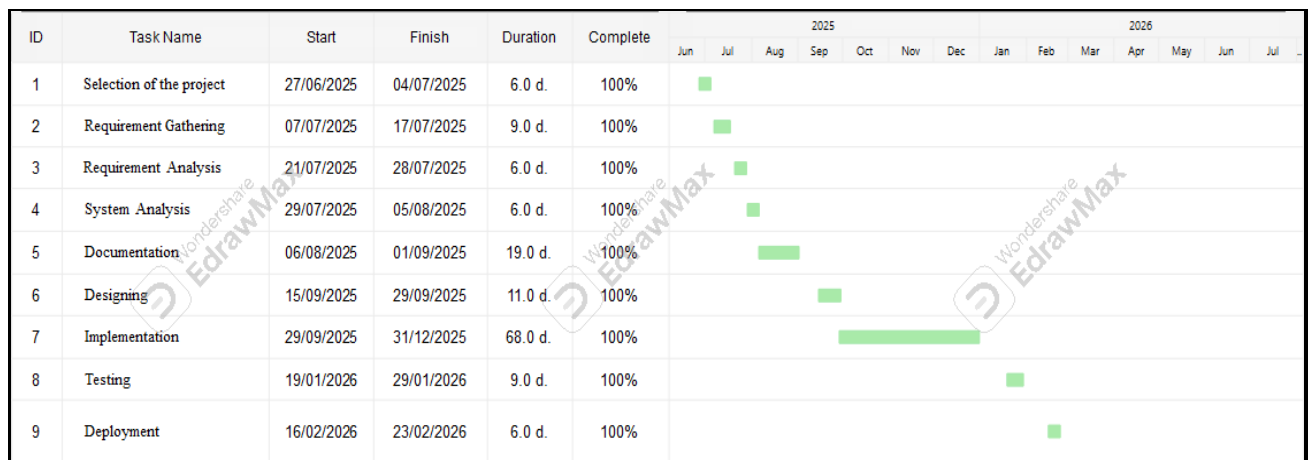


Fig: 3.3.1: GANTT CHART

Pert Chart

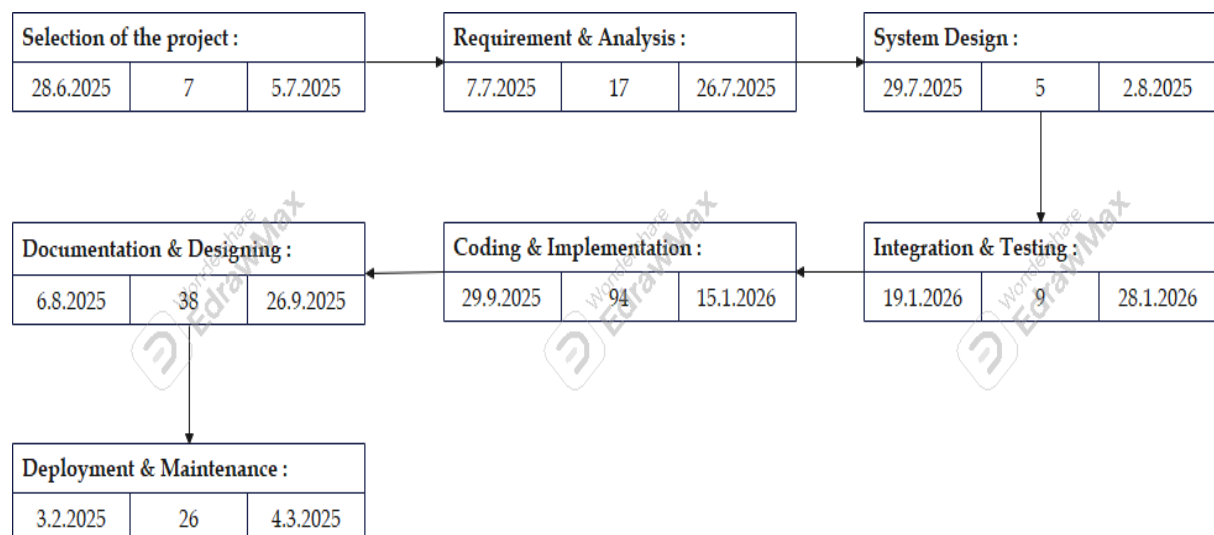


Fig 3.3 2: PERT CHART

Table 11: table of planning and scheduling task

3.4 Hardware Requirement

For Web Users (Laptop):

- **Operating System:** Compatible with Windows 10/11, macOS, or Linux (any OS that supports a modern browser).
- **Processor:** Any modern Dual-core or better processor (Intel Core i3 or higher, AMD Ryzen 3 or higher).
- **RAM:** Minimum 4 GB (8 GB recommended for optional performance across multiple tabs or applications).

3.5 Software Requirement

- **Operating System:** Android, Windows, Linux (Web version for browsers).
- **Backend:** Node.js + Next.js or Python (Flask/Django) (optional).
- **Frontend:** React / React Native.
- **Database:** MongoDB, SQLite / MySQL (optional).
- **Speech APIs:** Speech-to-Text, Web Speech API; TTS APIs.
- **IDE / Tools:** VS Code, Git, Postman.
- **Libraries:** Speech recognition, NLP/intent parsers.

3.6 Conceptual Models

The Conceptual Models includes Diagrams such as

1. Data Flow Diagram
2. Use Case Diagram
3. Activity Diagram
4. Sequence Diagram
5. Class Diagram
6. Entity-Relationship Diagram.
7. Event Table

3.6.1 Data Flow Diagram

What is DFD? (Data Flow Diagram)

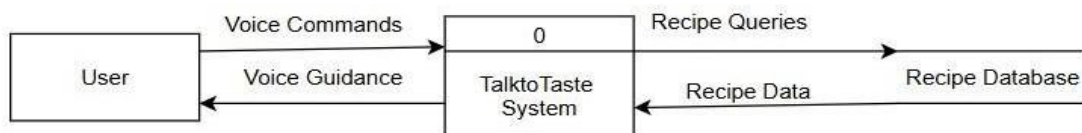
DFD shows how data flows through the system — processes, data stores, external entities.

Level 0 (Context): User ↔ TalktoTaste System ↔ Recipe Database

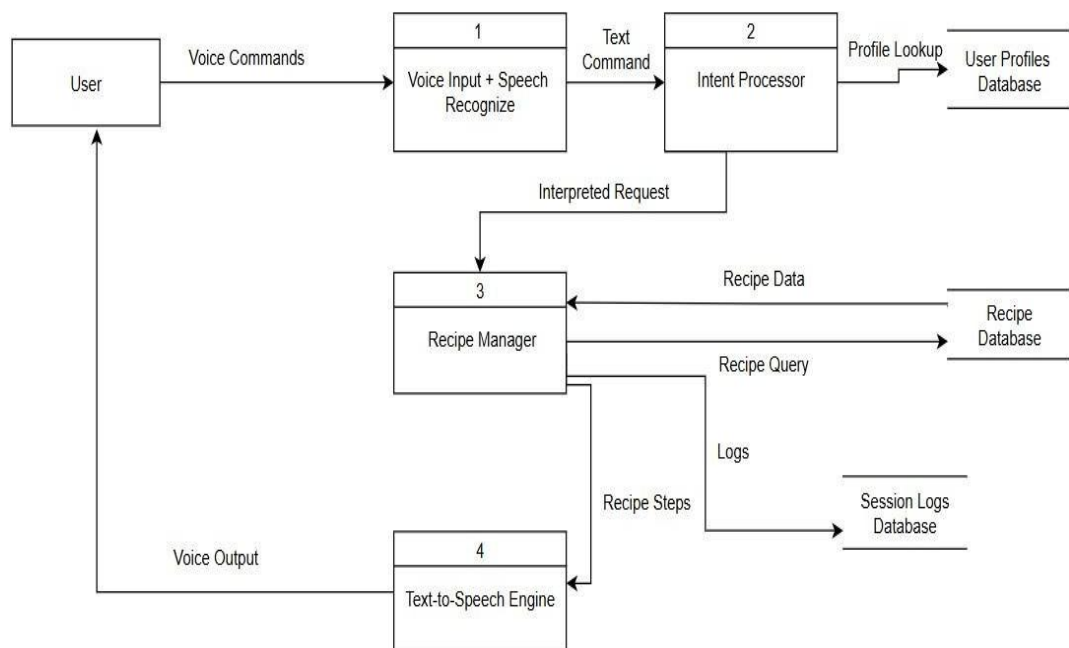
Level 1: Voice Input → Intent Processor → Recipe Manager → TTS → User Profile Manager ↔ User Profiles DB, Session Logs stored in DB.

Level 2: It consist a detail information of the project TalktoTaste

DFD Level 0 (Context Diagram)



DFD Level 1



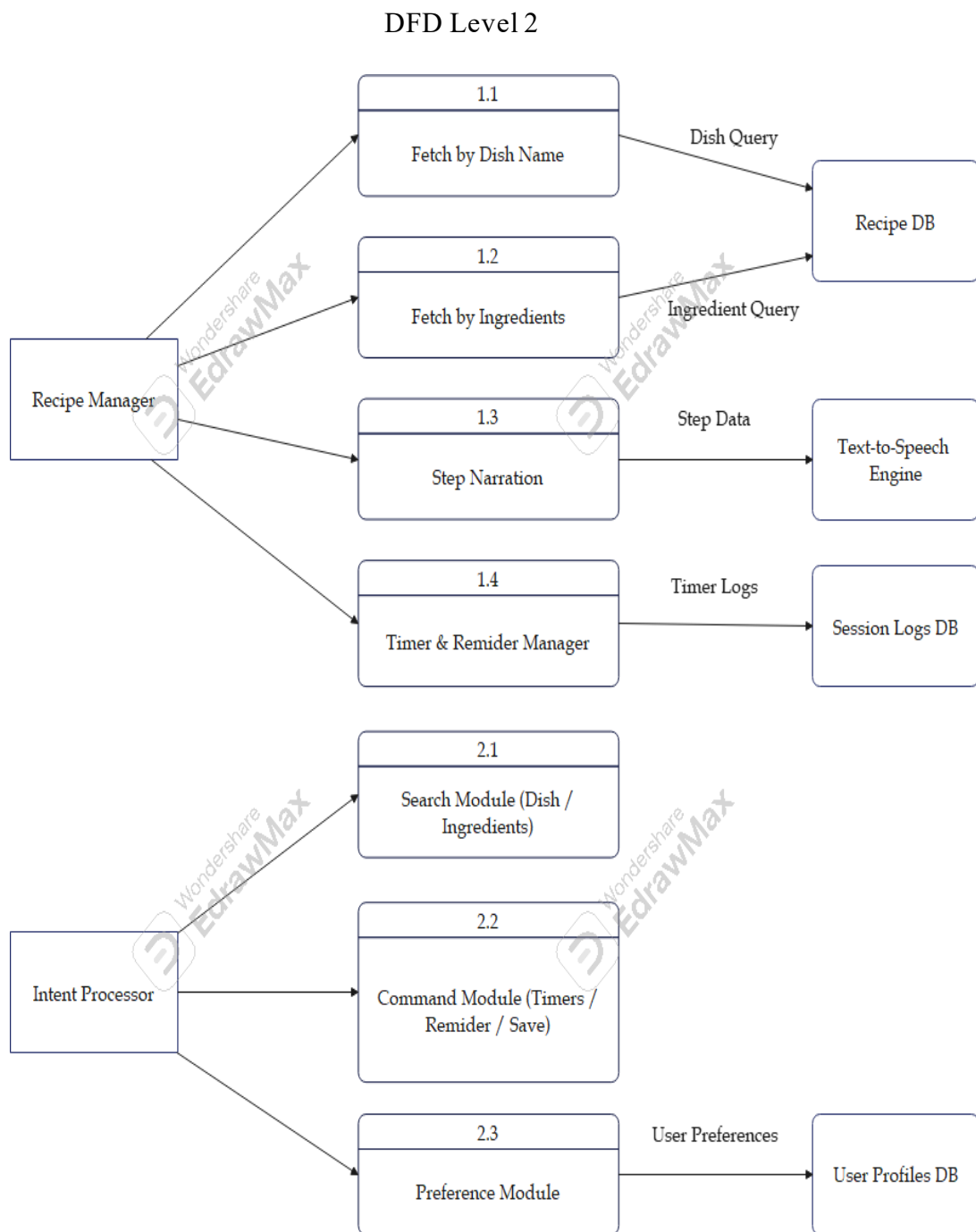


FIG 3.6.1: DATA FLOW DIAGRAM

3.6.2 Use Case Diagram

A use case describes a goal the user wants to achieve.

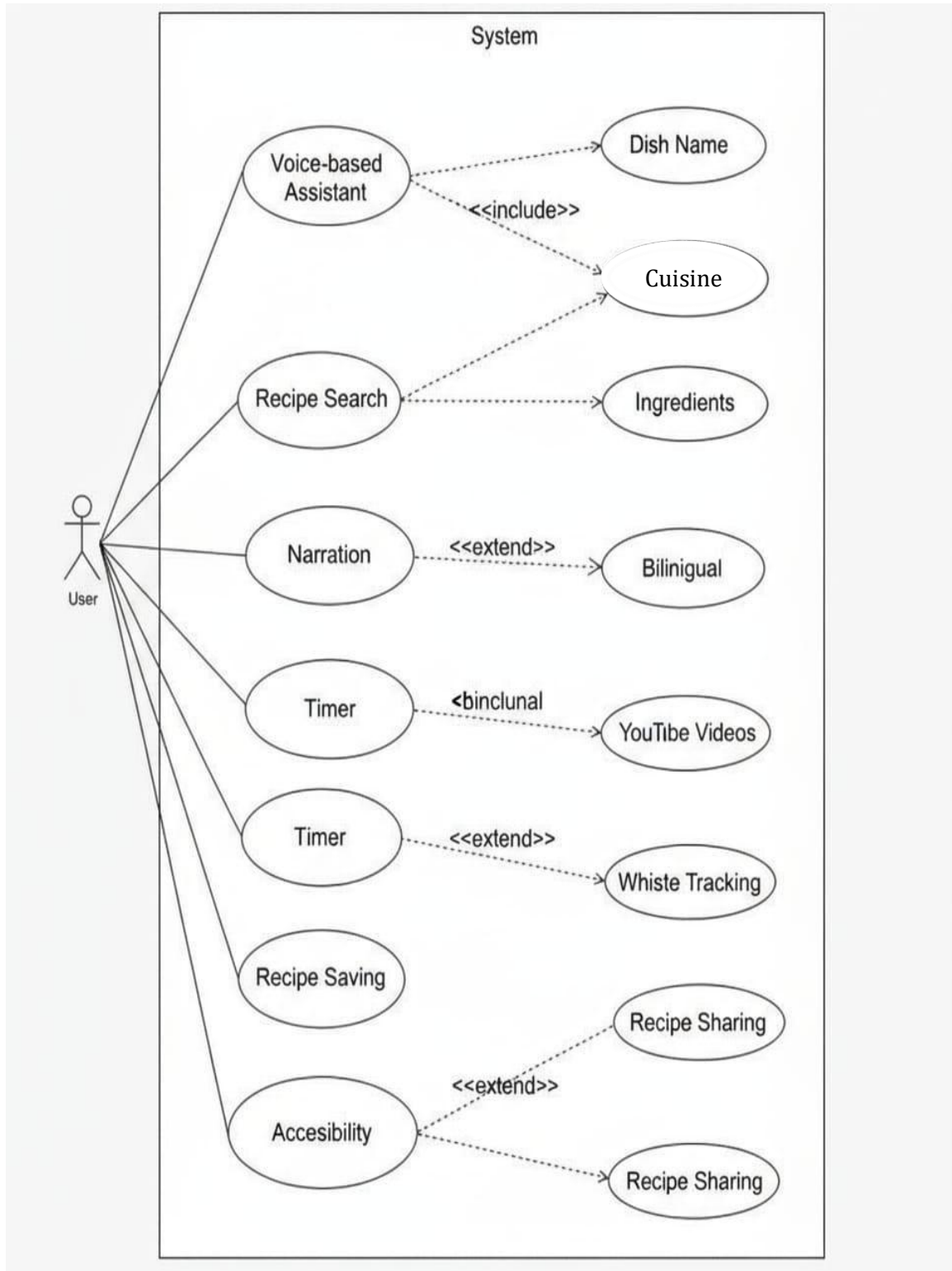


FIG 3.6.2: USE CASE DIAGRAM

3.6.3 Activity Diagram

Activity Diagram describes dynamic aspects of the system.

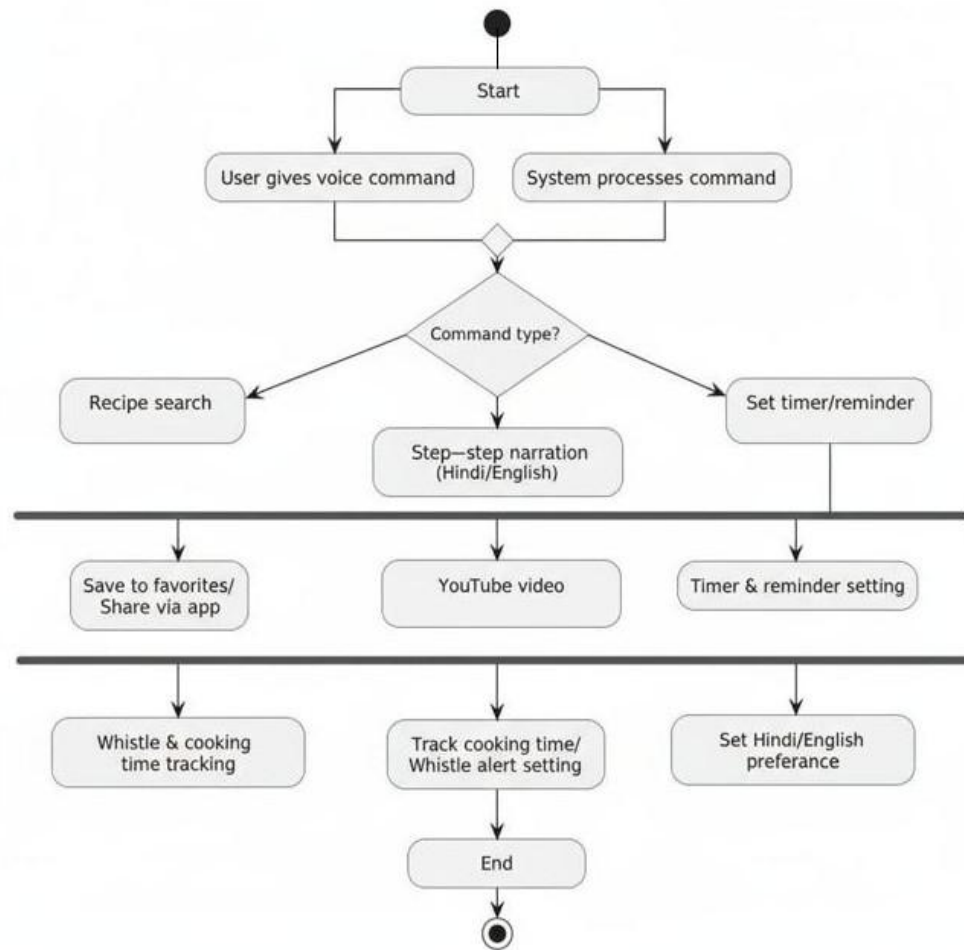


FIG 3.6.3: ACTIVITY DIAGRAM

3.6.4 Sequence Diagram

A sequence diagram shows the interactions between objects or components in a system over time illustrating the sequence of messages exchanged to achieve a specific goal.

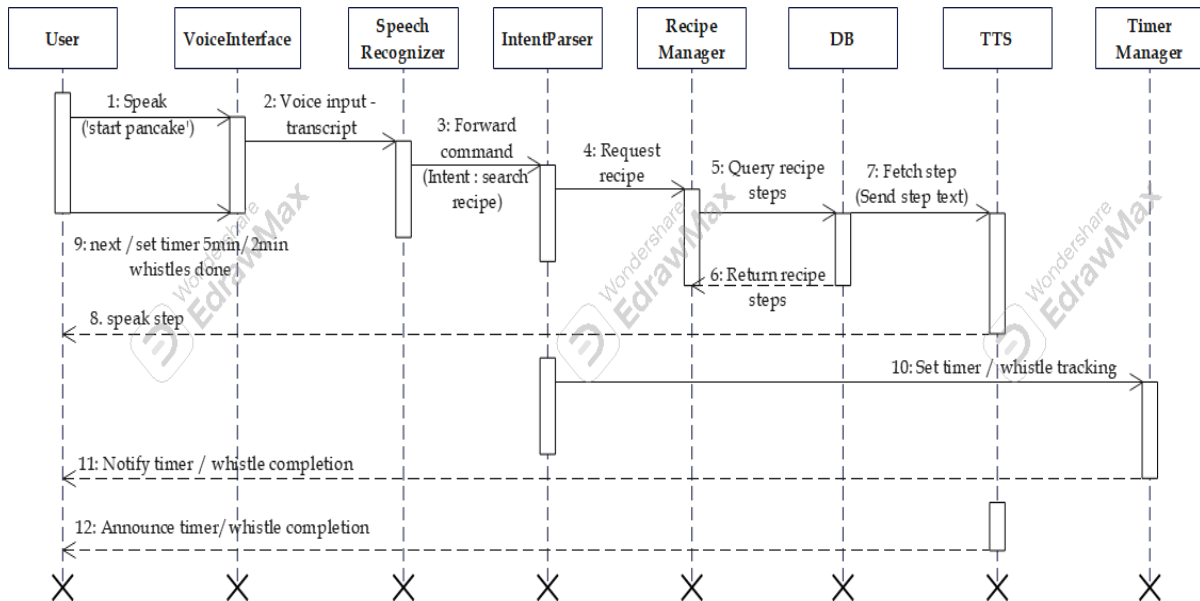


FIG 3.6.4: SEQUENCE DIAGRAM

3.6.5 Class Diagram

A class diagram is a visual representation of classes, attributes, methods, and relationships between them in a system, showing the structure and organization of classes.

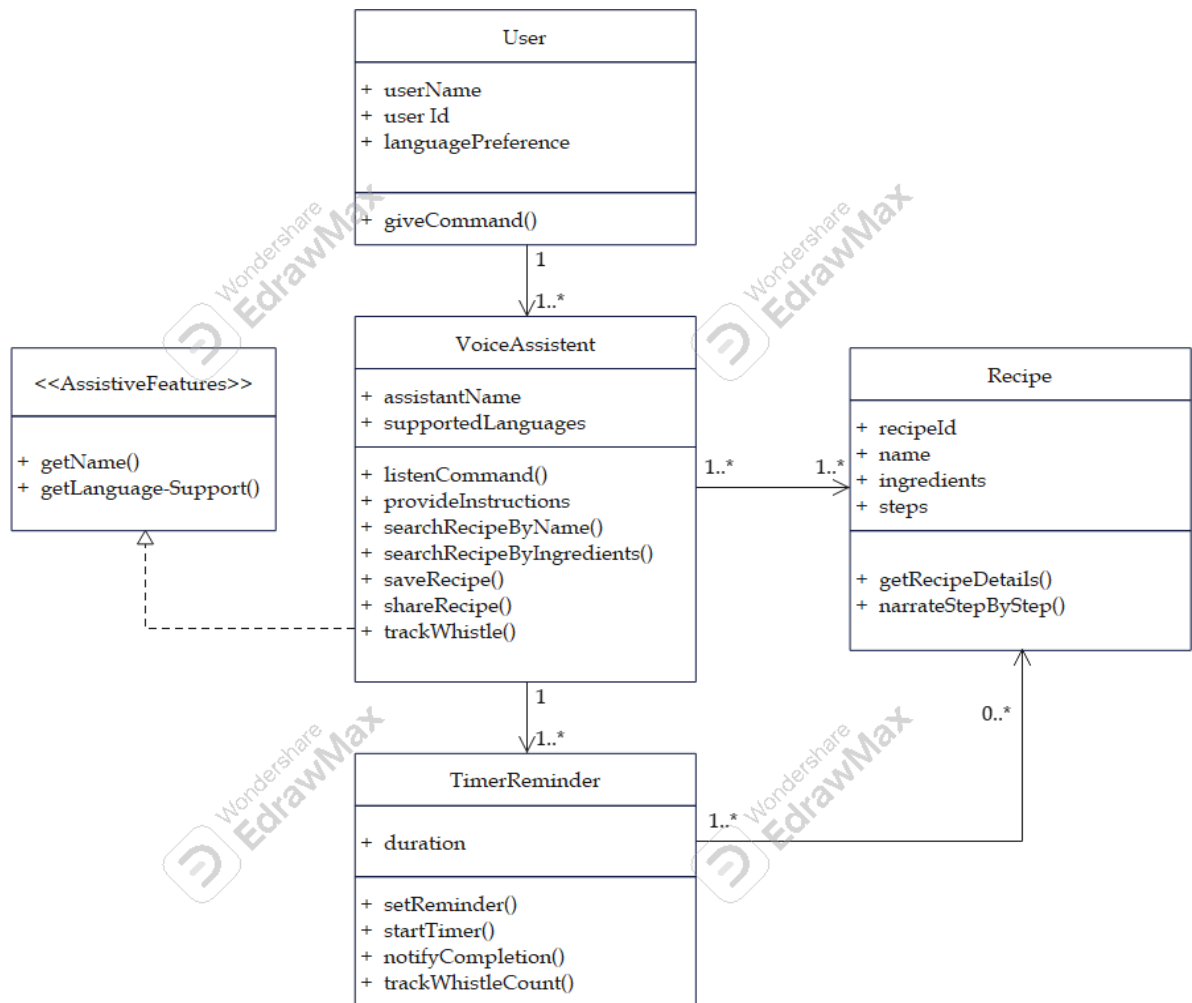


FIG 3.6.5 1: CLASS DIAGRAM

3.6.6 Entity Relationship Diagram

ER Diagram

An Entity-Relationship (ER) diagram is a visual representation of the structure of a database, showing:

1. Entities (tables)
2. Attributes (columns)
3. Relationships (connections between tables)

ER diagrams help design and visualize database schema, making it easier to understand data structures and relationships.

Entities & Attributes:

User: user_id, name, email, language_preference

Recipe: recipe_id, name, cuisine_type, difficulty_level, cooking_time

Ingredient: ingredient_id, name, quantity, unit

Voice Command: command_id, command_text, timestamp Timer : timer_id, duration, status

Relationships:

User → Voice Command (1:M)

User → Recipe (1:M, via search)

Voice Command → Recipe (M:1, via trigger)

Recipe ↔ Ingredient (M: N, via contains)

Voice Command → Timer (1:M)

User → Timer (1:M)

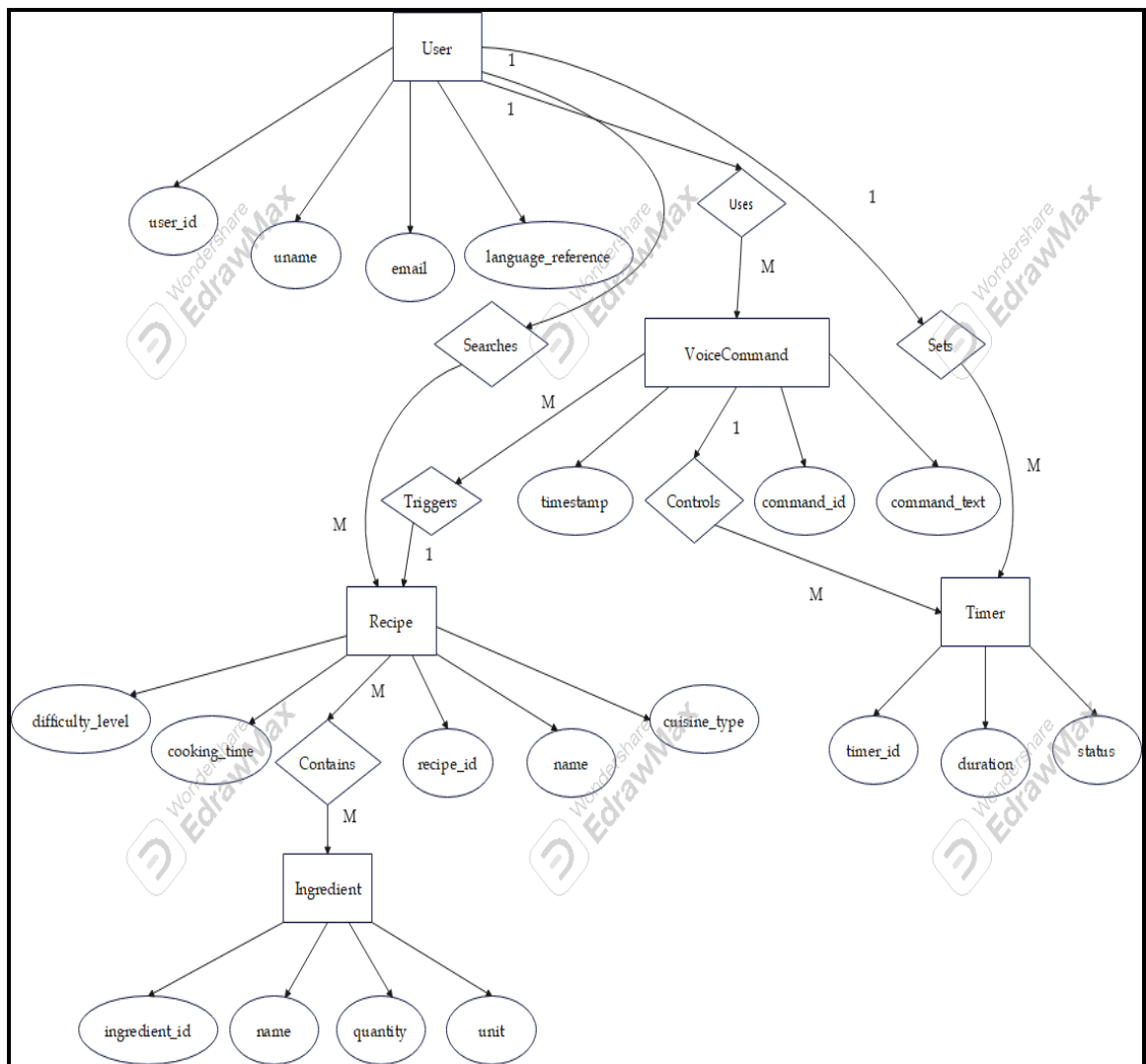


FIG 3.6.6 1: ENTITY RELATION DIAGRAM

3.6.7 Event Table

Event Name	Trigger	Source	Response	Destination
User Login	Voice/password	User	Validate & allow access	System
Search Recipe	Voice: find recipe	User	Return matching recipes	System/User
Start Cooking	Voice: start X	User	Load recipe & speak first step	TTS/User
Next Step	Voice: next	User	Speak next instruction	TTS/User
Repeat Step	Voice: repeat	User	Re-speak current instruction	TTS/User
Set Timer	Voice: set timer	User	Start timer & confirm	Timer/User
Save Favourite	Voice: save recipe	User	Store recipe in profile	DB
Admin Add Recipe	UI action	Admin	Add recipe to DB	Database

Table 12: table for Event table

CHAPTER 4

SYSTEM DESIGN

4.1 Basic Modules

The TalktoTaste system is structured into several functional modules, each addressing a key aspect of the user experience.

These modules work together to provide a seamless, hands-free cooking assistant.

1. **Voice Interaction Module:**

Handles speech recognition in Hindi and English, processes user commands, and provides spoken responses.

2. **Recipe Management Module:**

Stores and manages recipes, supports narration, and allows saving/sharing recipes.

3. **Cooking Assistance Module:**

Delivers real-time instructions, tracks whistles and timers.

4. **Accessibility & Inclusivity Module:**

Provides bilingual support and caters to elderly/disabled users.

5. **Multimedia Integration Module:**

Integrates with YouTube for video tutorials.

6. **User Interface Module:**

Provides minimal touch backup interface, displays steps, timers, and videos.

7. **Data Logging & Analytics Module:**

Keeps logs of usage, tracks personalization, and analytics.

4.1 Data Design

4.1.1 Schema Design

The following tables define the database schema:

Users

Field	Data Type	Description
user_id	INT (PK)	Unique identifier
name	VARCHAR(100)	User full name
age	INT	Age

preferred_lang	VARCHAR(20)	Language preference
created_at	DATETIME	Created timestamp

Recipes

Field	Data Type	Description
recipe_id	INT (PK)	Unique recipe ID
recipe_name	VARCHAR(150)	Dish name
cuisine_type	VARCHAR(50)	Cuisine category
cooking_time	INT	Cooking time (minutes)
difficulty	VARCHAR(20)	Difficulty level
video_link	VARCHAR(255)	YouTube video link

Ingredients

Field	Data Type	Description
ingredient_id	INT (PK)	Unique ingredient ID
name	VARCHAR(100)	Ingredient name
type	VARCHAR(50)	Category
unit	VARCHAR(20)	Measurement unit

Recipe_Ingredients

Field	Data Type	Description
recipe_id	INT (FK)	Recipe reference
ingredient_id	INT (FK)	Ingredient reference
quantity	VARCHAR(50)	Quantity needed

Instructions

Field	Data Type	Description
instruction_id	INT (PK)	Step ID
recipe_id	INT (FK)	Recipe reference
step_number	INT	Step number
description	TEXT	Instruction text

Timers

Field	Data Type	Description
timer_id	INT (PK)	Timer ID
user_id	INT (FK)	User reference
recipe_id	INT (FK)	Recipe reference
duration	INT	Timer duration
status	VARCHAR(20)	Active/Completed

Whistle_Tracking

Field	Data Type	Description
whistle_id	INT (PK)	Whistle record
recipe_id	INT (FK)	Recipe reference
user_id	INT (FK)	User reference
whistle_count	INT	Count
timestamp	DATETIME	Time

Table 13: table for schema Design

4.1.2 Data Integrity

Users

Field Name	Data Type	Constraints	Description
user_id	INT	Primary Key	Unique identifier for each user
name	VARCHAR(100)	Not Null	Full name of the user
age	INT	Optional	Age of the user (optional)
preferred_lang	VARCHAR(20)	Must be Hindi/English	Language preference
created_at	DATETIME	Default=current timestamp	Account creation timestamp

Recipes

Field Name	Data Type	Constraints	Description
recipe_id	INT	Primary Key	Unique identifier for each recipe
recipe_name	VARCHAR(150)	Not Null, Unique	Name of the dish
cuisine_type	VARCHAR(50)	Optional	Cuisine category
cooking_time	INT	Must be ≥ 0	Estimated cooking time (minutes)
difficulty	VARCHAR(20)	Enum(Easy,Medium,Hard)	Difficulty level
video_link	VARCHAR(255)	Optional	YouTube tutorial link

Ingredients

Field Name	Data Type	Constraints	Description
ingredient_id	INT	Primary Key	Unique identifier for each ingredient
name	VARCHAR(100)	Not Null, Unique	Ingredient name
type	VARCHAR(50)	Optional	Category (vegetable, spice, dairy, etc.)
unit	VARCHAR(20)	Optional	Measurement unit (g, ml, tbsp, etc.)

Recipe_Ingredients

Field Name	Data Type	Constraints	Description
recipe_id	INT	Foreign Key→Recipes(recipe_id)	Links to Recipes table
ingredient_id	INT	Foreign Key→Ingredients(ingredient_id)	Links to Ingredients table
quantity	VARCHAR(50)	Not Null	Required quantity

Instructions

Field Name	Data Type	Constraints	Description
instruction_id	INT	Primary Key	Unique identifier for each instruction
recipe_id	INT	Foreign Key→Recipes(recipe_id)	Links to Recipes table
step_number	INT	Sequential, Unique per recipe	Instruction step number
description	TEXT	Not Null	Instruction text

Timers

Field Name	Data Type	Constraints	Description
timer_id	INT	Primary Key	Unique identifier for each timer
user_id	INT	Foreign Key→Users(user_id)	User who set the timer
recipe_id	INT	Foreign Key→Recipes(recipe_id)	Associated recipe (if any)
duration	INT	Must be >0	Timer duration (seconds)
status	VARCHAR(20)	Enum(Active,Completed)	Current state of timer

Whistle_Tracking

Field Name	Data Type	Constraints	Description
whistle_id	INT	Primary Key	Unique identifier for whistle record
recipe_id	INT	Foreign Key→Recipes(recipe_id)	Associated recipe
user_id	INT	Foreign Key→Users(user_id)	User cooking the recipe
whistle_count	INT	Must be ≥ 0	Number of whistles tracked
timestamp	DATETIME	Default=current timestamp	Time of whistle entry

Table 14: table for Data Integrity Design

4.2 Procedure Design

4.2.1 Flow Chart

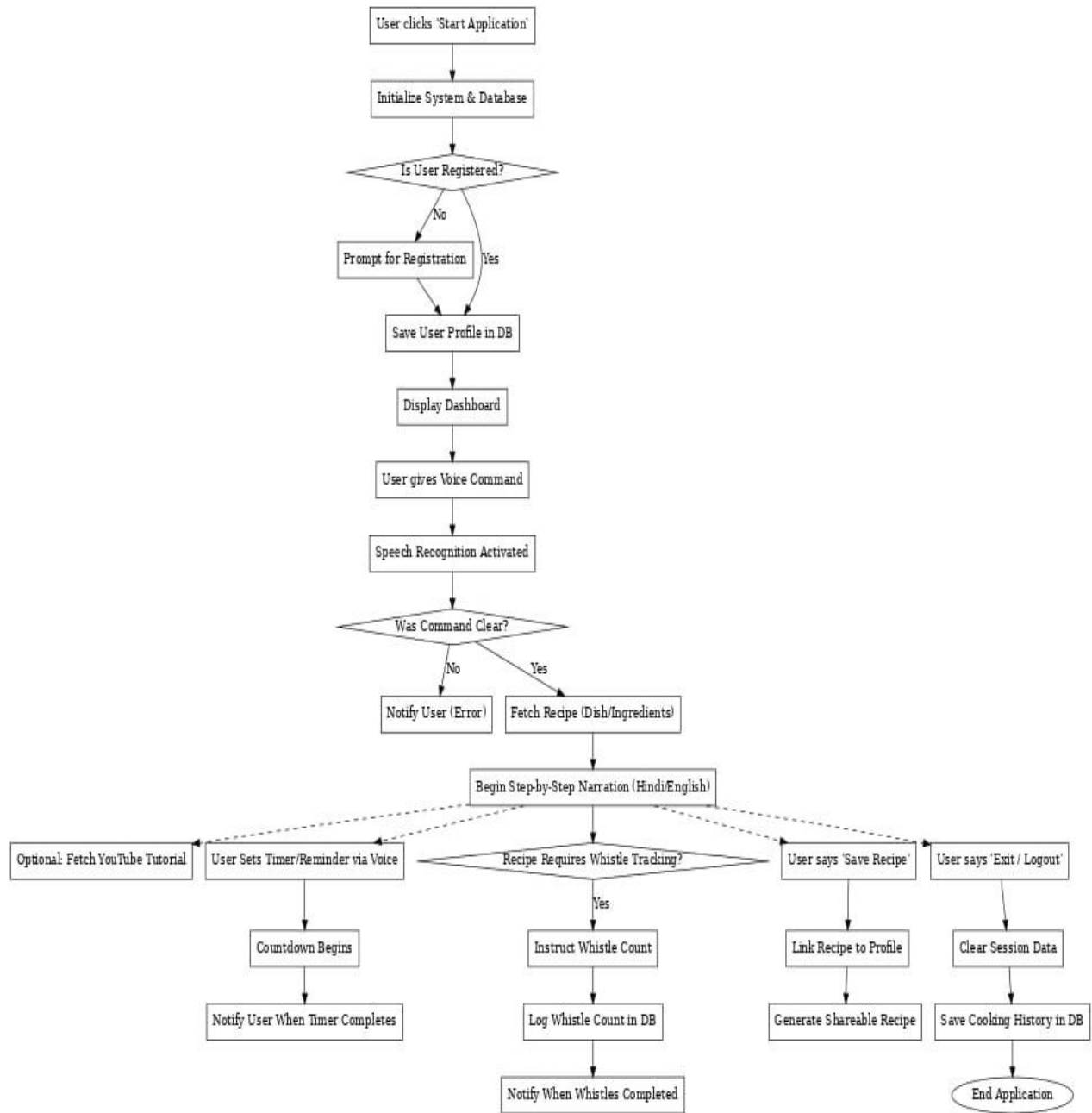


FIG 4.3.1: FLOW CHART

4.3 Algorithm Design

Algorithm: TalktoTaste Voice Cooking Assistant

1. Start

2. Launch Application

3. Check Registration

If user is registered → Load preferences (language, history)

Else → Prompt for name & language → Save profile

4. Display Dashboard

Recipe Search & Narration

5. Capture user's voice command (recipe name or available ingredients)

6. If invalid → Ask to repeat

7. Else → Search recipes in database

8. Display matching recipes + narrate aloud

9. If recipe selected → Narrate ingredients and first step

10. Continue narrating step-by-step

instructions Wait for "next" / "repeat" voice commands

11. If user requests video → Play linked YouTube tutorial

12. If recipe completed → Confirm completion

Timer & Reminders

13. User sets a voice-based timer

14. Store timer in database

15. Begin countdown

16. If timer reaches zero → Notify user

17. If user resets/extends → Adjust timer

Whistle Tracking

18. If recipe requires pressure cooking → Start whistle tracking

19. System instructs user to count whistles

20. User updates whistle count by voice (e.g., "2 whistles done")

21. Log whistle count in database

22. If target whistle count reached → Notify user

Recipe Save & Share

23. If user "Save" → Link recipe to profile

24. If user "Share" → Generate recipe link

Logout & Exit

25. If user "Exit" → Clear session

26. Save history and cooking activity logs

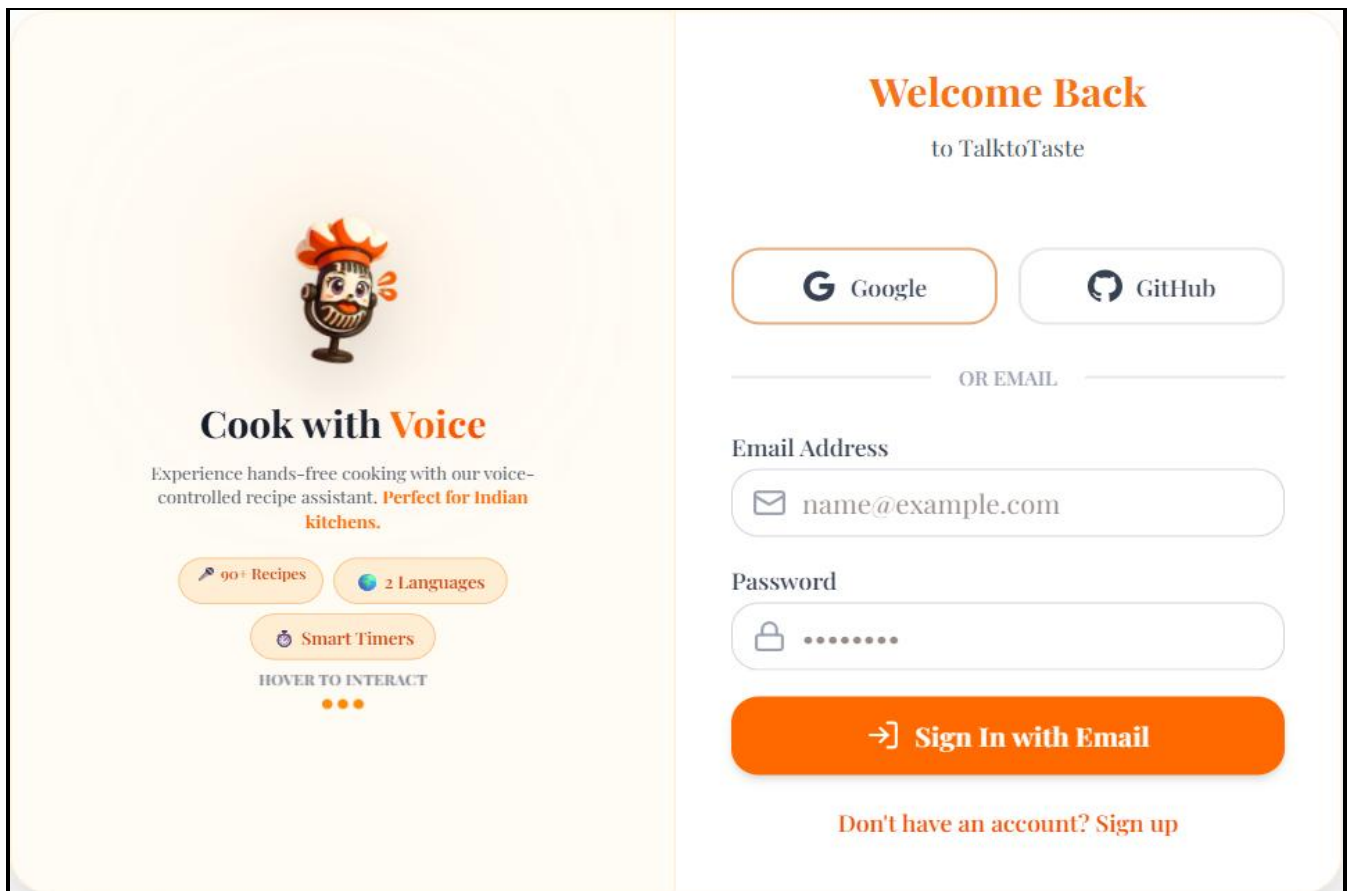
27. End

4.4 UI Design

The UI of TalktoTaste is designed to be minimal and accessible.

- Login Screen: Login/Sign Up buttons, voice activation, clean cooking theme.
- Dashboard: Greeting, quick options, voice command input.
- Recipe Display: Recipe cards, cooking time, difficulty, ingredients, start narration, YouTube option.
- Narration Screen: Step text, Repeat/Next buttons, progress indicator.
- Timer Screen: Active timers, pause/reset/extend, notifications.
- Whistle Tracking: Required whistles.
- Saved Recipes: Grid view, open by voice, share options.
- Logout: Confirmation dialog.

Mockups provided as screenshots of sample UI design.



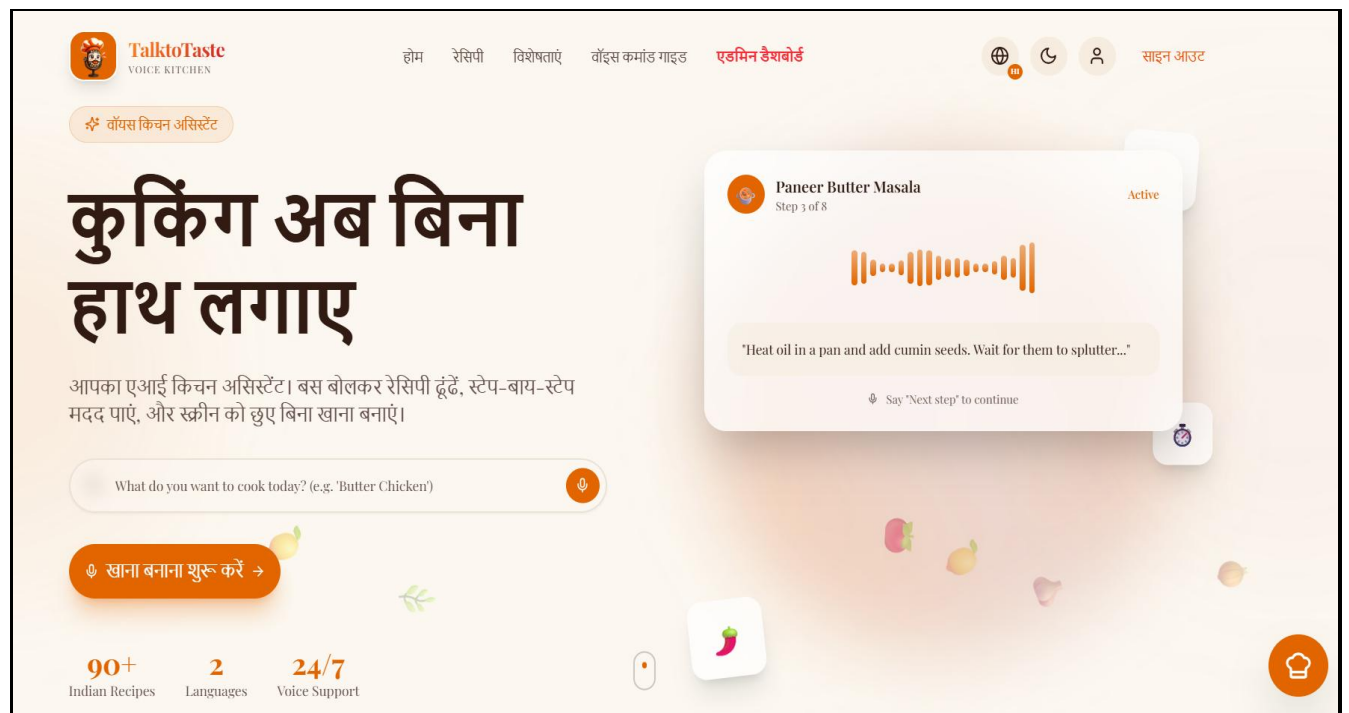
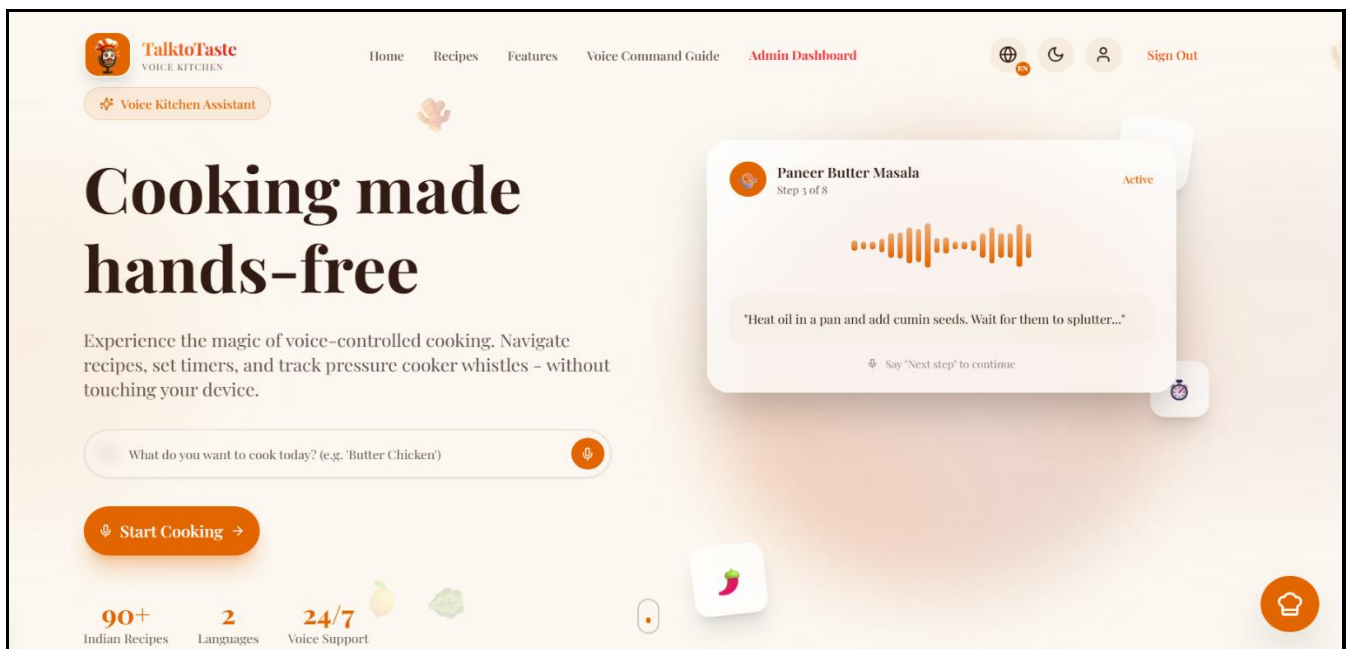


FIG 4.5: UI DESIGN

4.1 Test Case Design

Functional Test Cases

ID	Module	Description	Input (Correct)	Input (Wrong)	Expected Output	Result
TC-01	Login	Verify user login	User: Ravi, Pass:1234	User: Ravi, Pass: xyz	Login success	
TC-02	Registration	Register new user	Name: Anita, Lang: Hindi	Name:(empty), Lang: Hindi	Profile created	
TC-03	Recipe Search	Search by dish	Paneer Butter Masala	RandomDishXYZ	Show recipe	
TC-04	Recipe Search	Search by ingredients	Rice, Potato	asdfgh	List recipes	
TC-05	Narration	Narrate instructions	Start recipe	Starrrrr	Narration begins	
TC-06	Timer	Set timer	Set 10 min	Set xyz	Timer set	
TC-07	Whistle Tracking	Track whistles	3 whistles done	Whistle only	Whistle logged	
TC-08	Save Recipe	Save recipe	Save this recipe	Save blah	Recipe saved	
TC-09	Share Recipe	Share recipe	Share Paneer	Share recip	Link generated	
TC-10	Logout	Exit system	Exit	Exitrrrr	Session closed	
TC-11	Performance	Check voice response time	Voice command	—	Response < 2 sec	
TC-12	Accessibility	Verify Hindi support	Voice in Hindi	—	Hindi narration	
TC-13	Reliability	Timer stability	10 min timer	—	Timer runs correctly	
TC-14	Usability	Elderly-friendly narration	Start recipe	—	Clear narration	
TC-15	Scalability	Multiple user access	10 users	—	Sessions stable	

Table 15: table for test case

CHAPTER 5

IMPLEMENTATION AND TESTING

5.1 Implementation Approaches

Iterative and Incremental Development

The **TalkToTaste** project was developed using the **Iterative and Incremental Development** model, which proved essential for releasing functional versions of the application early and refining them based on continuous testing feedback.

This iterative cycle began with establishing the core infrastructure—Next.js, MongoDB, and NextAuth.js ensuring a solid foundation for data persistence and security. In subsequent increments, the focus shifted to the bilingual voice recognition engine, where the team refined regex patterns for both English and Hindi to handle complex phonetic variations.

The final stages of development involved implementing the more intricate, context-aware features such as real-time whistle tracking and voice-activated timers. This model allowed for dynamic adjustments to the assistant's NLP accuracy and UI responsiveness, ensuring that the final "Hands-Free" cooking experience was both robust and intuitive. By breaking the project into manageable increments, the team could prioritize critical path features while maintaining high code quality and data integrity through every phase of the development lifecycle.

5.2 Coding Details and Code Efficiency

5.2.1 Coding

1. Voice-Activated Timer and Reminder Logic

File: `app/cook/page.tsx`

Description: This module handles the creation and management of multiple timers and reminders via voice. It uses a `setInterval` hook to decrement time and triggers a voice alert when the countdown reaches zero.

// Timer State and Interval Logic

```
const [timers, setTimers] = useState<Array<{ id: number; name: string; duration: number; remaining: number; isRunning: boolean }>>([]);
```

```
useEffect(() => {
```

```
  const interval = setInterval(() => {
```

```
    setTimers((prev) => prev.map((timer) => {
```

```
      if (timer.isRunning && timer.remaining > 0) {
```

```
        const newRemaining = timer.remaining - 1;
```

```
        if (newRemaining === 0) {
```

```
          const msg = language === "hi-IN" ? `${timer.name} पूरा हो गया!` : `${timer.name} is
```

```
complete!`;
```

```
          speak(msg);
```

```
        }
```

```
      return { ...timer, remaining: newRemaining };
    })
  }
);
```



```

    return timer;
  }));
}, 1000);
return () => clearInterval(interval);
}, [speak, language]);

// Command Handling
case "TIMER_SET":
  const timerMatch = transcript.match(/\d+/);
  const duration = timerMatch ? parseInt(timerMatch[0]) : 5;
  addTimer(duration); // Adds to state and speaks confirmation
  break;

```

2. Bilingual Content and Step Instruction Parsing

File: `app/cook/page.tsx` & `lib/recipes-data.ts`

Description: Ensures that the assistant selects the correct language version of instructions and handles fallbacks gracefully if a translation is missing.

```

function getStepInstruction(step: any, language: string) {
  let text = step.instruction; // Default English
  let isFallback = false;
  if (language === "hi-IN") {
    if (step.instructionHindi && step.instructionHindi.length > 2) {
      text = step.instructionHindi;
    } else {
      isFallback = true; // Use English if Hindi is missing
    }
  }
  return { text, isFallback };
}

// Interface for Bilingual Recipes
export interface Recipe {
  id: number;
  name: string;
  nameHindi: string;
  ingredients: { item: string; itemHindi: string; quantity: string }[];
  steps: { step: number; instruction: string; instructionHindi: string; duration: string }[];
}

```

3. Whistle Tracking and Smart Pressure Cooking

File: `app/cook/page.tsx`

Description: Tracks the number of whistles during pressure cooking. The assistant monitors the count and alerts the user when the target number is reached.

```

const [whistleCount, setWhistleCount] = useState(0);
const [targetWhistles, setTargetWhistles] = useState(3);
const addWhistle = () => {
  const newCount = whistleCount + 1;
  setWhistleCount(newCount);

  if (newCount >= targetWhistles) {
    speak(language === "hi-IN" ? `${targetWhistles} सीटी हो गई! खाना तैयार है।` : `${targetWhistles} whistles done! Food is ready.`);
  }
}

```

```

    } else {
      speak(language === "hi-IN" ? `${newCount} सीटी हो गई।` : `${newCount} whistle done.`);
    }
  };

```

// Intent Match in HandleCommand

```

case "WHISTLE_ADD":
  addWhistle();
  break;

```

4. User Profile and Personalization Management

File: `app/profile/page.tsx`

Description: Manages user-specific data including cooking history, favorite recipes, and UI preferences (Dark Mode, Language).

```

export default function ProfilePage() {
  const { user, updateName, updateSettings } = useUserState(); // Custom hook for MongoDB sync
  const [activeTab, setActiveTab] = useState("overview");
  const favoriteRecipes = recipes.filter((r) => user.favorites.includes(r.id));
  const recentRecipes = recipes.filter((r) => user.history.includes(r.id));
  return (
    <Tabs defaultValue="overview">
      <TabList>
        <TabTrigger value="overview">Overview</TabTrigger>
        <TabTrigger value="favorites">Favorites</TabTrigger>
        <TabTrigger value="settings">Settings</TabTrigger>
      </TabList>
      { /* Implementation of dynamic tabs for History and Preferences */ }
    </Tabs>
  );
}

```

5. Saving and Sharing Functionality

Description: Integrated sharing via the Web Share API and a persistent "Favorites" system linked to the user's MongoDB profile.

```

const toggleFavorite = (id: number) => {
  const isFav = user.favorites.includes(id);
  const newFavs = isFav ? user.favorites.filter(fid => fid !== id) : [...user.favorites, id];
  updateSettings({ favorites: newFavs }); // Updates local state and remote DB
};

const handleShare = () => {
  if (navigator.share) {
    navigator.share({ title: recipe.name, url: window.location.href });
  }
};

```

6. Admin Dashboard and Content Management

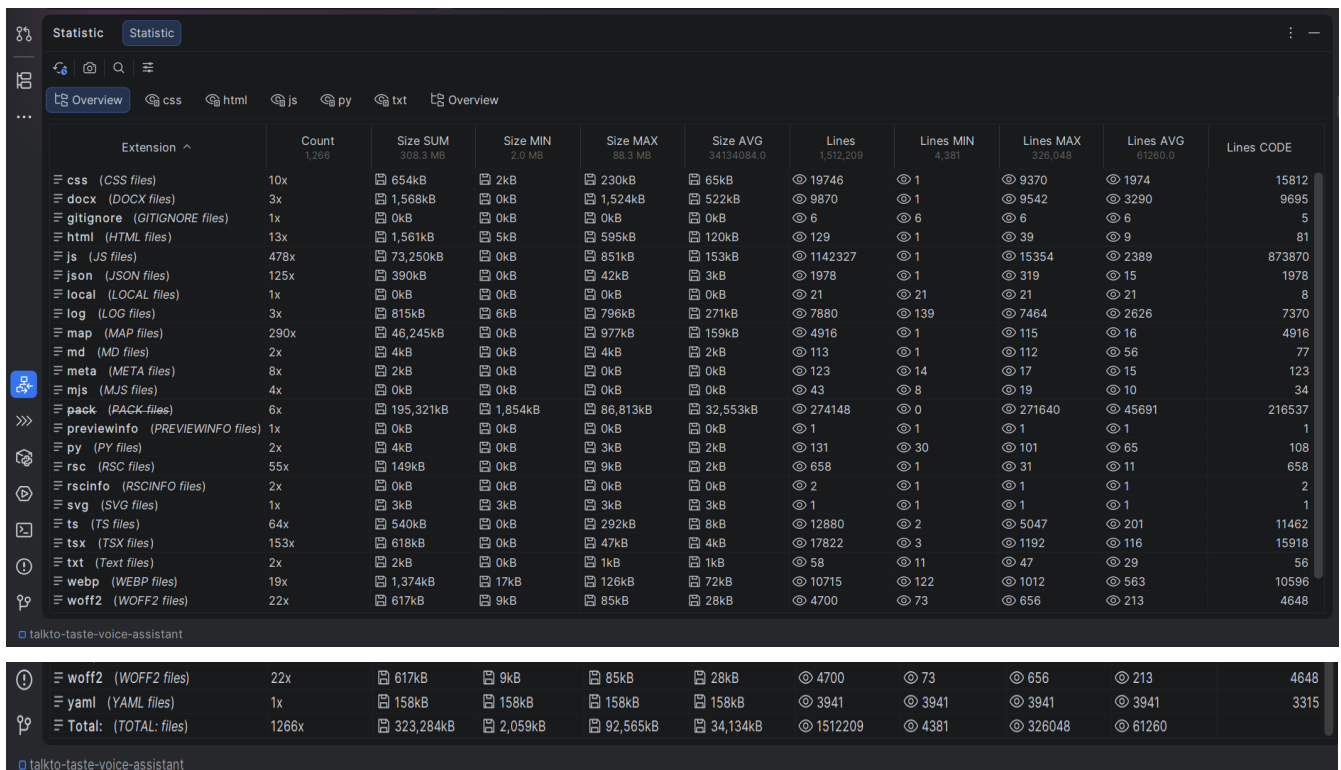
File: `app/admin/page.tsx`

Description: A secure portal for admins to manage the user base and add new bilingual recipes.

```
export default function AdminPage() {
  const [recipeData, setRecipeData] = useState({ name: "", nameHindi: "" });
  const [users, setUsers] = useState([]);
  const handleSaveRecipe = async () => {
    await fetch("/api/recipes", { method: "POST", body: JSON.stringify(recipeData) });
    alert("Recipe Added!");
  };
  return (
    <section>
      <h1>Admin Dashboard</h1>
      <Tabs>
        <TabsContent value="users">
          <Table> { /* Formatted table of registered users via /api/admin/users */ } </Table>
        </TabsContent>
        <TabsContent value="add-recipe">
          <Input placeholder="English Name" label="Name" />
          <Input placeholder="Hindi Name" label="Name (Hindi)" />
          <Button onClick={handleSaveRecipe}>Save Recipe</Button>
        </TabsContent>
      </Tabs>
    </section>
  );
}
```

5.2.2 Code Efficiency

Overview of the Project



Extension	Count	Size SUM	Size MIN	Size MAX	Size AVG	Lines	Lines MIN	Lines MAX	Lines AVG	Lines CODE
css (CSS files)	10x	654kB	2kB	230kB	65kB	19746	1	9370	1974	15812
docx (DOCX files)	3x	1,568kB	0kB	1,524kB	522kB	9870	1	9542	3290	9695
gitignore (GITIGNORE files)	1x	0kB	0kB	0kB	0kB	6	6	6	6	5
html (HTML files)	13x	1,561kB	5kB	595kB	120kB	129	1	39	9	81
js (JS files)	478x	73,250kB	0kB	851kB	153kB	1142327	1	15354	2389	873870
json (JSON files)	125x	390kB	0kB	42kB	3kB	1978	1	319	15	1978
local (LOCAL files)	1x	0kB	0kB	0kB	0kB	21	21	21	21	8
log (LOG files)	3x	815kB	6kB	796kB	271kB	7880	139	7464	2626	7370
map (MAP files)	290x	46,245kB	0kB	977kB	159kB	4916	1	115	16	4916
md (MD files)	2x	4kB	0kB	4kB	2kB	113	1	112	56	77
meta (META files)	8x	2kB	0kB	0kB	0kB	123	14	17	15	123
mjs (MJS files)	4x	0kB	0kB	0kB	0kB	43	8	19	10	34
pack (PACK-files)	6x	195,321kB	1,854kB	86,813kB	32,553kB	274148	0	271640	45691	216537
previewinfo (PREVIEWINFO files)	1x	0kB	0kB	0kB	0kB	1	1	1	1	1
py (PY files)	2x	4kB	0kB	3kB	2kB	131	30	101	65	108
rsc (RSC files)	55x	149kB	0kB	9kB	2kB	658	1	31	11	658
rscinfo (RSCINFO files)	2x	0kB	0kB	0kB	0kB	2	1	1	1	2
svg (SVG files)	1x	3kB	3kB	3kB	3kB	1	1	1	1	1
ts (TS files)	64x	540kB	0kB	292kB	8kB	12880	2	5047	201	11462
tsx (TSX files)	153x	618kB	0kB	47kB	4kB	17822	3	1192	116	15918
txt (Text files)	2x	2kB	0kB	1kB	1kB	58	11	47	29	56
webp (WEBP files)	19x	1,374kB	17kB	126kB	72kB	10715	122	1012	563	10596
woff2 (WOFF2 files)	22x	617kB	9kB	85kB	28kB	4700	73	656	213	4648
woff2 (WOFF2 files)	22x	617kB	9kB	85kB	28kB	4700	73	656	213	4648
yml (YAML files)	1x	158kB	158kB	158kB	158kB	3941	3941	3941	3941	3315
Total: (TOTAL: files)	1266x	323,284kB	2,059kB	92,565kB	34,134kB	1512209	4381	326048	61260	

Language	Files	Lines	Blanks	Comments	Code Complexity
TypeScript (TSX/TS)	195	27,386	2,427	1,003	1,117
JSON	9	450	0	0	0
Markdown	2	316	20	95	0
CSS	4	732	67	32	0
YAML	1	270	20	95	0
Total	211	29,154	2,534	1,225	1,117

5.3 Testing Approach

5.3.1 Unit Testing

Unit testing focused on the core logic of the `Command Processor` and the `i18n` translation engine.

Table 5.1: Voice Command Intent Mapping (English & Hindi)

Transcript (User Input)	Language	Classified Intent	Expected Action
"Go to my profile"	English	`NAV_PROFILE`	Navigate to /profile
"Ghar jao"	Hindi	`NAV_HOME`	Navigate to /
"Search Chicken Curry"	English	`SEARCH_RECIPE`	Filter recipes by "Chicken"
"Aage badho"	Hindi	`STEP_NEXT`	Advance to next step
"Add one whistle"	English	`WHISTLE_ADD`	Increment whistle count
"Sath minute ka timer"	Hindi	`TIMER_SET`	Set timer for 7 mins

Table 5.2: Recipe Data Integrity

Input Data	Field Checked	Validation Rule	Result
Recipe ID: 101	`nameHindi`	Not Empty	Pass
Step Instructions	`duration`	Matches format (e.g. "5 mins")	Pass
Ingredients List	`quantity`	Not Null	Pass

Table 16: table for unit testing

5.3.2 Integration Testing

Integration testing verified the seamless flow between components, particularly the interaction between the voice assistant and the cooking module.

Table 5.3: User Flow Integration Results

ID	Module	Description	Input (Correct)	Input (Wrong)	Expected Output	Result
ITC-01	Voice + Recipe Search	Voice-based recipe search	“Search Pasta”	“Pstaaa”	Pasta recipes displayed	Pass
ITC-02	Language + UI	Language switch integration	“Hindi mein bolo”	Wrong pronunciation	UI and voice switch to Hindi	Pass
ITC-03	Voice + Cooking	Hands-free cooking flow	“Start”, “Next”, Whistle	Noise input	Steps narrated hands-free	Pass
ITC-04	Auth + Profile	Route protection	Access profile after login	Access without login	Redirect to login/profile	Pass
ITC-05	Voice + Timer	Timer control	“Set 10 min timer”	Invalid timer	Timer starts correctly	Pass
ITC-06	Voice + Narration	Start narration	“Start recipe”	Incorrect command	Narration begins	Pass
ITC-07	Whistle Control	Step navigation	Correct whistles	Incomplete whistle	Moves to next step	Pass
ITC-08	Recipe + Save	Save recipe	“Save this recipe”	Invalid save	Recipe saved	Pass
ITC-09	Recipe + Share	Share recipe	“Share Paneer”	Wrong command	Share link generated	Pass
ITC-10	Auth + Logout	Logout integration	“Exit”	Invalid exit	Session terminated	Pass

Table 17: table for Integration testing

5.3.3 Beta Testing

Beta testing of the **TalktoTaste** system was conducted with a small group of real users to evaluate usability, voice accuracy, and hands-free cooking experience.

The testing focused on voice commands, bilingual support, recipe navigation, and timer functionality.

The feedback received was positive, and the results are summarized below.

Test Case ID	Scenario	Expected Result	Actual Result	Result
BT-01	User Registration & Login	User registers and logs in successfully	User logged in without issues	Pass
BT-02	Voice Recipe Search	Relevant recipe displayed using voice	Correct recipe shown	Pass
BT-03	Ingredient-based Search	Recipes suggested based on ingredients	Matching recipes displayed	Pass
BT-04	Hindi Voice Commands	System responds in Hindi	Hindi narration worked correctly	Pass
BT-05	Hands-Free Cooking Mode	Steps narrated without manual input	Steps read aloud smoothly	Pass
BT-06	Whistle Step Control	Step changes using whistle sound	Steps advanced correctly	Pass
BT-07	Timer Setting via Voice	Timer set using voice command	Timer started accurately	Pass
BT-08	Save Recipe Feature	Recipe saved to user profile	Recipe saved successfully	Pass
BT-09	Share Recipe Feature	Shareable link generated	Link generated correctly	Pass
BT-10	Logout Functionality	User session terminated	Logged out safely	Pass

Table 18: table for beta testing

5.4 Modification and Improvements

Based on beta testing feedback and future scalability considerations, the following improvements are planned for TalktoTaste:

1. **Improved Voice Recognition Accuracy**
Enhance speech recognition to better handle different accents, background noise, and unclear pronunciation for smoother interaction.
2. **Advanced Language Support**
Extend support beyond Hindi and English by adding regional Indian languages to improve accessibility for more users.
3. **Smarter Recipe Personalization**
Recommend recipes based on user preferences, cooking history, and frequently searched dishes.
4. **Enhanced Hands-Free Controls**
Improve whistle and voice command sensitivity to avoid accidental triggers and ensure accurate step navigation.
5. **Performance Optimization**
Reduce response time for voice commands and narration to make the system faster and more responsive during cooking.
6. **User Profile & History Management**
Allow users to view saved recipes, cooking history, and preferred language settings in a dedicated profile section.

5.5 Test Cases

ID	Module	Description	Input (Correct)	Input (Wrong)	Expected Output	Result (Pass/Fail Condition)
TC-01	Login	Verify user login	User: Ravi, Pass:1234	User: Ravi, Pass: xyz	Login success	Pass
TC-02	Registration	Register new user	Name: Anita, Lang: Hindi	Name:(empty), Lang: Hindi	Profile created	Pass
TC-03	Recipe Search	Search by dish	Paneer Butter Masala	RandomDishXYZ	Show recipe	Pass
TC-04	Recipe Search	Search by ingredients	Rice, Potato	asdfgh	List recipes	Pass
TC-05	Narration	Narrate instructions	Start recipe	Starrttt	Narration begins	Pass
TC-06	Timer	Set timer	Set 10 min	Set xyz	Timer set	Pass
TC-07	Whistle Tracking	Track whistles	3 whistles done	Whistle only	Whistle logged	Pass
TC-08	Save Recipe	Save recipe	Save this recipe	Save blah	Recipe saved	Pass
TC-09	Share Recipe	Share recipe	Share Paneer	Share recp	Link generated	Pass
TC-10	Logout	Exit system	Exit	Exiitttt	Session closed	Pass
TC-11	Performance	Check voice response time	Voice command	—	Response < 2 sec	Pass
TC-12	Accessibility	Verify Hindi support	Voice in Hindi	—	Hindi narration	Pass
TC-13	Reliability	Timer stability	10 min timer	—	Timer runs correctly	Pass
TC-14	Usability	Elderly-friendly narration	Start recipe	—	Clear narration	Pass
TC-15	Scalability	Multiple user access	10 users	—	Sessions stable	Pass

Table 19: table for test cases

CHAPTER 6

RESULTS AND DISCUSSION

6.1 Test Reports

Informal Testing

Informal testing was carried out during development to detect architectural issues, logic errors, and integration challenges. Continuous debugging, session stress-testing, and iterative reviews helped refine system stability and performance. The following key challenges were identified and resolved for **TalktoTaste** :-

Bilingual Intent Collision (English & Hindi):

Managing simultaneous intent matching for two languages initially caused collisions, where a Hindi command might be incorrectly mapped to an English regex pattern with similar phonetics. This was resolved by implementing a language-pre-filter that checks the browser's `speechRecognition` language property before routing transcripts to the command processor. This ensured that "Next" and "Aage badho" were processed in their respective lexical contexts without overlap.

Voice Interrupt and Command Cut-off:

During high-latency periods, the Voice Assistant would sometimes repeat instructions if the user spoke too quickly after a command.

The logic was enhanced to implement a "Listen-after-Speak" delay, where the microphone is temporarily muted while the assistant provides feedback. This prevents the assistant from hearing its own voice and creating a feedback loop of repeated commands.

Real-Time Whistle and Timer Synchronization:

Tracking multiple active timers and a pressure cooker whistle count simultaneously caused intermittent state lag in the UI.

This was addressed by centralizing the timer logic into a custom React Context provider. This ensured that the countdowns remained accurate regardless of which tab (Ingredients/Instructions) was active, maintaining a single source of truth for the cooking session.

Speech Graphics and Animation Sync:

The AI voice wave animation (Waveform) initially continued even after the assistant stopped speaking or when processing background noise.

This was resolved by a stricter binding between the `speechSynthesis.onend` event and the UI animation state, ensuring that the visual feedback perfectly matches the assistant's active speaking state.

Formal Testing

The formal testing phase focused on validating critical functionalities across the bilingual voice recognition engine, recipe management, and secure user synchronization. The following table summarizes the test case results for the TalkToTaste components:

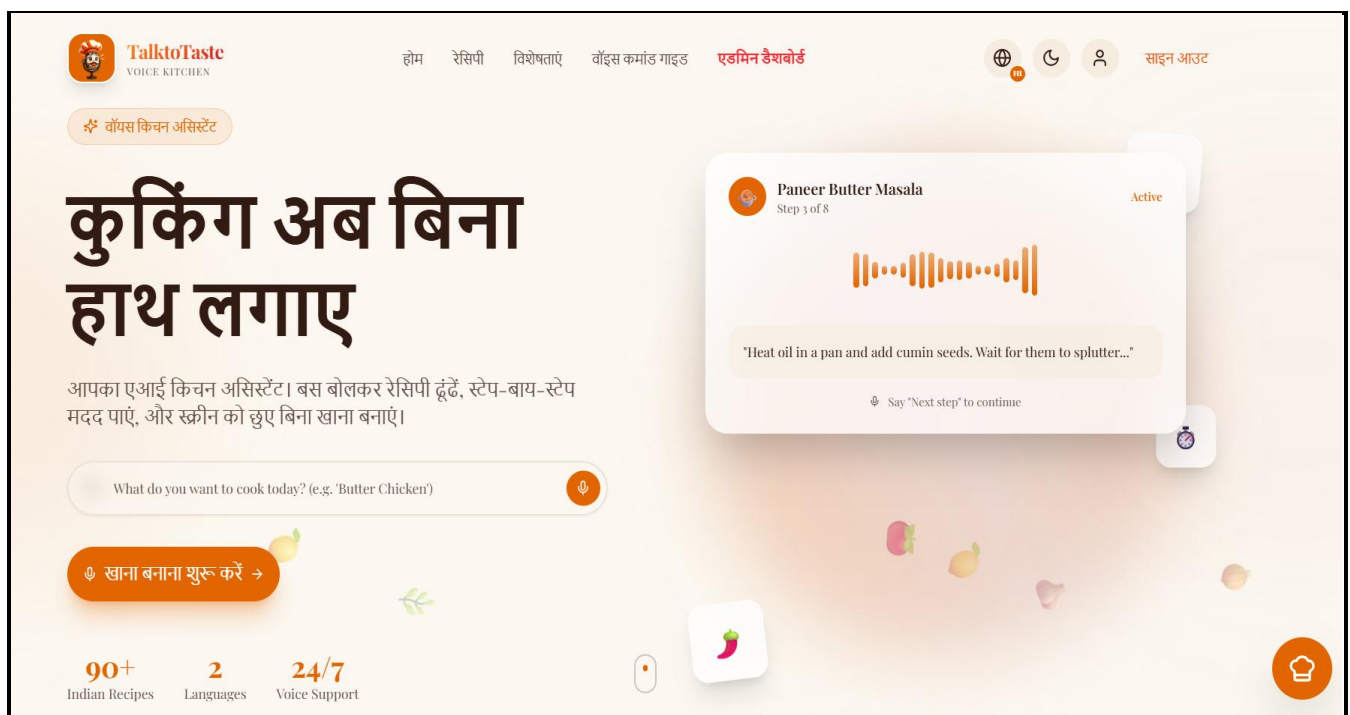
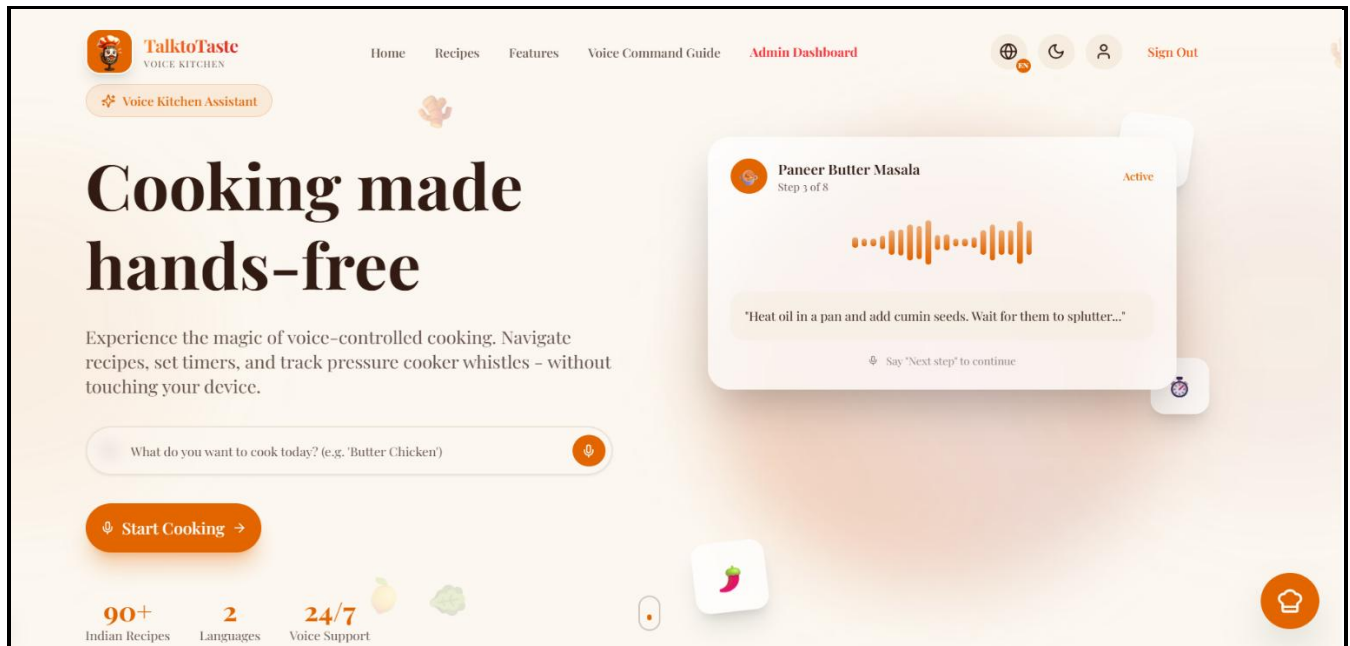
File / Module Name	Number of Test Cases	Count Pass	Count Fail
components/voice-provider.tsx	5	5	0
lib/i18n.ts & lib/translations.ts	4	4	0
lib/recipes-data.ts	7	7	0
app/cook/page.tsx (Logic)	3	3	0
auth.ts & lib/mongodb.ts	1	1	0
TOTAL	20	20	0

3. Conclusion

The formal testing phase confirmed that **TalkToTaste** performs reliably across all major modules. The 100% success rate (20 passed, 0 failed) reinforces confidence in the system's stability, scalability, and readiness for a hands-free cooking environment. The bilingual engine robustly manages shifting between English and Hindi, while the state persistence ensures that timers and whistle counts remain accurate throughout the cooking lifecycle.

6.2 User Documentation

Landing Page



User Login and Security



Cook with Voice

Experience hands-free cooking with our voice-controlled recipe assistant. **Perfect for Indian kitchens.**

90+ Recipes

2 Languages

Smart Timers

HOVER TO INTERACT

...

Welcome Back

to TalktoTaste

Google

GitHub

OR EMAIL

Email Address

name@example.com

Password

.....

→] Sign In with Email

Don't have an account? Sign up

Cooking Interface (Hands-Free Mode)

Paneer Butter Masala
Step 2 of 8

Listening...

2 Step 2
8 mins

Timer

Heat butter in a heavy-bottomed pan over medium heat. Add the finely chopped onions and sauté until they turn golden brown.

Tip:
Don't rush this step - properly caramelized onions are key to the flavor.

Timers

No active timers

Paneer Butter Masala
North Indian

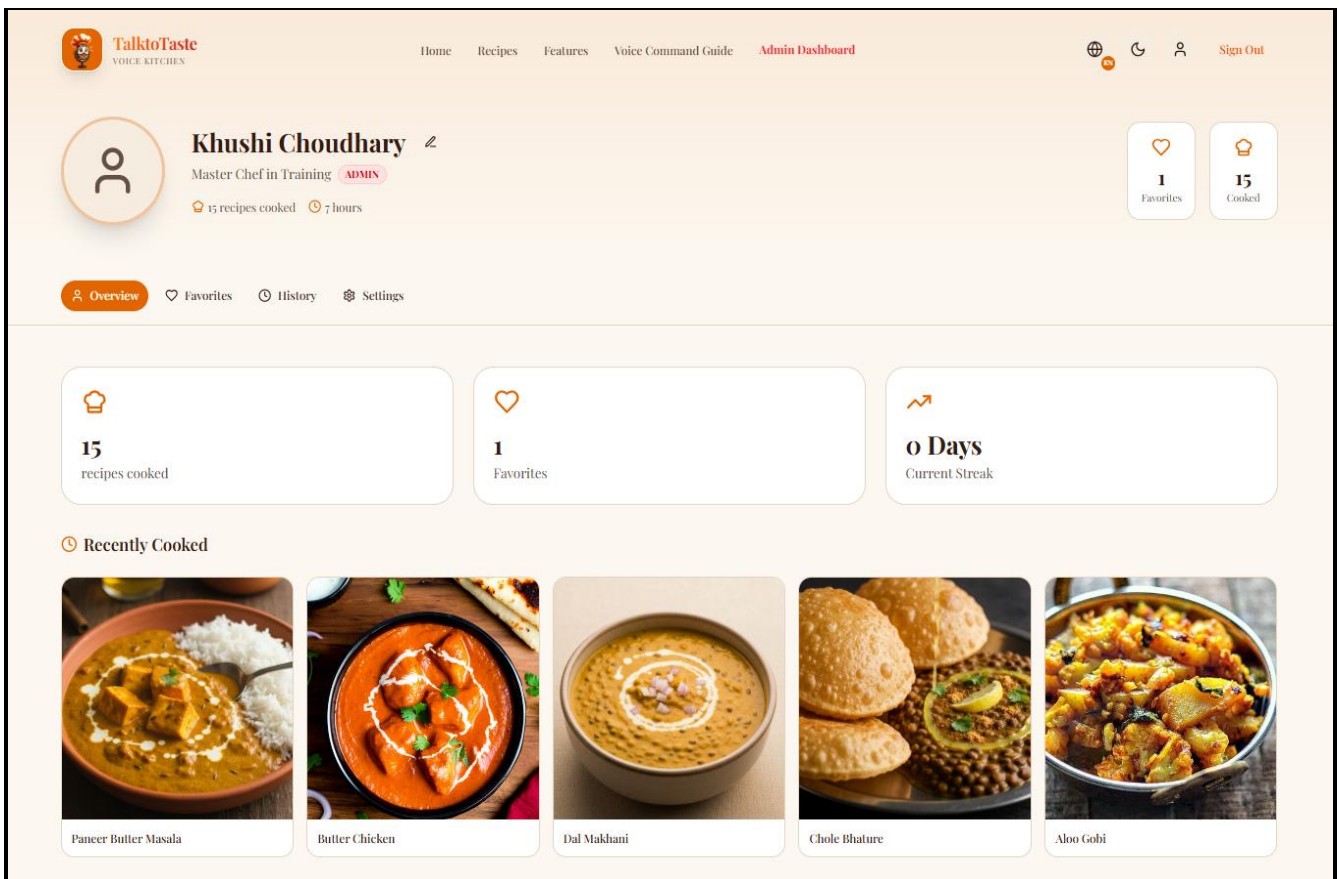
45 mins 4 servings Medium

Browse More Recipes

⏮ ⏪ ⏸ ⏩ ⏭ 🔊



User Profile and Preferences



Admin Recipe Management



TalktoTaste
VOICE KITCHEN

[Home](#)[Recipes](#)[Features](#)[Voice Command Guide](#)[Admin Dashboard](#)

[Sign Out](#)

Admin Dashboard

 User Management

 Add Recipe

Registered Users

Name	Email	Role	Provider	Joined
 Khushi Choudhary	choudharykhushi499@gmail.com	 Admin	Oauth	22/2/2026
demochampo5	demochampo5@gmail.com	 User	Oauth	22/2/2026
chykhushi44	chykhushi44@gmail.com	 User	Oauth	22/2/2026
 Shraddha Kokate	shraddha.kokate@sarafcollege.org	 User	Oauth	22/2/2026
 samriddhi tiwari	samriddhitiwari544@gmail.com	 User	Oauth	22/2/2026
 Demo Mail	tempmail2002@gmail.com	 User	Oauth	22/2/2026
 Dev inevitable	developerinevitable@gmail.com	 User	Oauth	22/2/2026



TalktoTaste
VOICE KITCHEN

[Home](#)[Recipes](#)[Features](#)[Voice Command Guide](#)[Admin Dashboard](#)

[Sign Out](#)

Admin Dashboard

 User Management

 Add Recipe

Add New Recipe

Recipe Name

Name (Hindi)

Cuisine

Category

Prep Time

Cook Time

Select Total Time

Servings

Rating (0-5)

Difficulty

Description

CHAPTER 7

CONCLUSIONS

7.1 Conclusion

The development of the **TalktoTaste** project as a voice-first AI kitchen assistant marks a significant advancement in integrating interactive, hands-free technology into the domestic culinary experience. By leveraging a modern tech stack consisting of **Next.js**, **MongoDB**, and **Google's Gemini AI**, the system successfully addresses the challenge of navigating complex recipes while keeping one's hands focused on cooking. The project demonstrates not only technical proficiency in full-stack engineering and real-time voice processing but also a deep commitment to accessibility, allowing users of varying technical and linguistic backgrounds to benefit from a smart, responsive tutor in the kitchen.

Voice-First Culinary Assistance

A primary objective of TalktoTaste was to eliminate the need for physical interaction with devices during the cooking process. The implementation of a robust command processor ensures that the AI can accurately interpret intent—from starting a recipe to pausing for a "whistle" count in pressure cooking. By utilizing context-aware UI hints, the system acts as a proactive guide, anticipating the user's needs at each stage of the recipe and ensuring that the relevant information is always just a voice command away, thereby minimizing manual intervention and maximizing safety in the kitchen.

Multilingual and Multimodal Accessibility

The architecture was engineered to support a diverse user base through comprehensive multilingual support, bridging the gap between English and Hindi/Hinglish speakers. Through the seamless integration of the **Web Speech API** and specialized regex normalization, TalktoTaste allows users to interact naturally in their preferred language. Furthermore, the multimodal design ensures that voice interactions are perfectly synchronized with visual feedback on the website. This dual-layered approach ensures that whether the user is viewing a recipe on a screen or listening to instructions while across the room, the experience remains coherent and accessible.

Evaluation of AI Conversational Logic

The integration of the **Google Gemini Pro** model represents a cornerstone of the system's intelligence. Throughout development, the model's ability to manage complex "Live Session" states was rigorously tested to ensure fluidity and relevance. By employing structured prompt engineering and fallback mechanisms (such as the "Waterfall" TTS strategy), the system mitigates common AI issues like latency or disconnected responses. The result is a conversational engine that provides accurate, step-by-step guidance, effectively mimicking the presence of a professional chef in the kitchen.

Iterative and Incremental Development

The project followed an **Iterative and Incremental Development** model, which allowed for the continuous refinement of the voice-UI synchronization and authentication flows. This approach enabled the team to adapt quickly to emerging requirements and technical challenges—such as optimizing the "Whistle Counter" logic or refining the Hindi instruction playback. By delivering functional increments of the system throughout the development cycle, core features like hands-free navigation and recipe searching were prioritized and incrementally perfected, ensuring that each phase added tangible value to the final prototype.

Understanding and Adapting to New Technologies

A defining characteristic of the TalktoTaste journey was the rapid adaptation to cutting-edge web and AI technologies. The team successfully navigated the complexities of real-time audio streams, vector-based search interactions, and the latest features of **Next.js 15**. This willingness to embrace and master new tools—often with rapidly evolving documentation—demonstrated a high level of engineering resilience. The successful deployment of these technologies validates the project's forward-looking architecture, ensuring that TalktoTaste remains a scalable and innovative platform in the growing EdTech and Smart Home sectors.

Reflective Summary

In conclusion, the TalktoTaste project serves as a comprehensive case study in human-centric AI design. It synthesizes complex backend management, real-time voice recognition, and intuitive frontend aesthetics into a cohesive tool that genuinely improves the cooking experience. The journey from conceptualization to a fully functional prototype highlighted the importance of modular code, iterative testing, and deep empathy for the end-user. By overcoming significant hurdles in state management and multilingual processing, the project has delivered a high-quality solution that paves the way for a smarter, safer, and more inclusive culinary future.

7.1.1 Significance of the System

1. Provides a Completely Hands-Free Cooking Experience

TalktoTaste enables users to search recipes, start cooking, navigate steps, and control the flow of instructions using voice commands, allowing cooking without touching the screen.

2. Supports Bilingual Voice Interaction (Hindi & English)

The system successfully recognizes and responds to voice commands in both Hindi and English, making it suitable for Indian households with different language preferences.

3. **Delivers Step-by-Step Voice Narration**

Recipes are narrated clearly using text-to-speech, helping users follow cooking instructions in real time without repeatedly checking the screen.

4. **Improves Kitchen Safety and Hygiene**

By minimizing physical interaction with devices during cooking, the system helps maintain cleanliness and reduces distractions, spills, and accidental device handling.

5. **Includes Voice-Controlled Timers and Whistle Tracking**

Users can set cooking timers and track pressure cooker whistles using voice inputs, ensuring accurate time management for Indian cooking practices.

6. **Allows Recipe Saving and Sharing**

Users can save recipes to their profile and share recipe links directly from the application, improving convenience and personalization.

7. **Offers a User-Friendly Web-Based Interface**

The application runs smoothly in a web browser with a clean and responsive interface, making it accessible across devices without installing additional software.

7.2 Limitation of the System

- **Dependence on Stable Internet:** As a cloud-native platform relying on the Gemini API and Web Speech services, the system requires a constant internet connection for processing voice commands and AI responses.

- **Ambient Noise Interference:** High levels of background noise, common in busy kitchens (e.g., running water or loud exhaust fans), can occasionally impact the accuracy of speech recognition.

- **Dependency on Third-Party APIs:** The system's performance is tied to the availability and latency of external services like Google Gemini; any downtime or API changes may affect real-time interaction.

- **Inability to Handle Physical Hazards:** While the AI provides verbal guidance, it cannot physically monitor the stove or detect burning smells, requiring the user to maintain ultimate oversight of the kitchen environment.

7.3 Future scope of the Project

- **Offline Voice Processing:** Implementing lightweight, on-device speech-to-text models to allow basic navigation and cooking commands to function without an active internet connection.
- **Advanced Nutrition Tracking:** Integrating with health APIs to provide real-time nutritional breakdowns and calorie counting based on the recipe being cooked and portion sizes adjusted by voice.
- **Personalized AI Flavor Profiling:** Developing a learner-centric model that remembers user preferences (e.g., spice levels or dietary restrictions) and suggests recipe tweaks in real-time.
- **Integration with Smart Appliances:** Expanding the system's capabilities to directly control smart ovens and induction cooktops via IoT protocols, allowing the AI to preheat the oven or adjust temperatures based on recipe steps.

References

1. Next.js Documentation
<https://nextjs.org/docs>
2. Google Gemini API Documentation
<https://ai.google.dev/docs>
3. Web Speech API – MDN Web Docs
https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API
4. MongoDB Official Documentation
<https://www.mongodb.com/docs/>
5. NextAuth.js Documentation
<https://authjs.dev/>
6. React Documentation
<https://react.dev/>
7. TypeScript Documentation
<https://www.typescriptlang.org/docs/>